GIÁO TRÌNH ĐÀO TẠO KỸ SƯ FRONTEND CHUYÊN SÂU

GIÁO TRÌNH CSS3

TẬP 2: BỐ CỤC HIỆN ĐẠI & NÂNG CAO

Chuyên Đề Box Model, Outline, Flexbox, CSS Grid, Responsive & Animations

■ CSS Box Model • ■ Flexbox & Grid • ■ Responsive Web • ■ Animations

TÀI LIỆU CHUYÊN SÂU TẬP TRUNG CSS3 – TẬP 2

Làm Chủ Mô Hình Hộp (Box Model) • Bố Cục Một Chiều Flexbox & Hai Chiều CSS Grid

Hệ thống hóa toàn diện phương pháp dàn trang hiện đại • Thiết kế Responsive thích ứng đa màn hình • Tối ưu hiệu năng render chuẩn kỹ sư UI/UX

ĐỘI NGŨ BIÊN SOẠN WEB

Tài liệu lưu hành nội bộ

NĂM 2026

Phiên bản Standalone Tập 2

ĐỘI NGŨ BIÊN SOẠN WEB

TÀI LIỆU LƯU HÀNH NỘI BỘ – CHUYÊN ĐỀ CSS3 TẬP 2

GIÁO TRÌNH CSS3

TẬP 2: BỐ CỤC HIỆN ĐẠI & NÂNG CAO

Box Model, Outline, Flexbox, Grid, Responsive & Animations

TẬP TRUNG BIÊN SOẠN & THỰC HÀNH CSS3 NÂNG CAO

Biên soạn chi tiết • Minh họa trực quan • Sẵn sàng tích hợp nội dung mới

NĂM 2026

Phiên bản Standalone Tập 2 – XeLaTeX Unicode

MỤC LỤC

Giáo Trình Chuyên Sâu CSS3 – Tập 2: Bố Cục Hiện Đại & Kỹ Thuật Chuyên Sâu

I CSS3 Nâng Cao – Mô Hình Hộp & Bố Cục Hiện Đại 1

Chương 1: Chuyên Đề Toàn Diện: CSS Box Model (Mô Hình Hộp) 2

1.1 Bản Chất Của Khái Niệm ”Box” & Cấu Trúc CSS Box Model 2

1.1.1 1. Khái Niệm Cốt Lõi: Mọi Phần Tử Đều Là Chiếc Hộp 2

1.1.2 2. Bản Chất Hoạt Động Khi Trình Duyệt Vẽ (Render) 2

1.1.3 3. Box Model Giúp Điều Khiển Những Yếu Tố Nào? 5

1.1.4 4. Tại Sao Bắt Buộc Phải Hiểu Rõ CSS Box Model? 5

1.2 Chuyên Sâu Tầng Content (Vùng Nội Dung Cốt Lõi) 6

1.2.1 1. Content Là Gì? 6

1.2.2 2. Kích Thước Của Vùng Content Dưới 2 Chế Độ Box-Sizing 6

1.2.3 3. Content Có Thể Hiển Thị Những Gì? 7

1.2.4 4. Cách `width` & `height` Điều Chỉnh Vùng Content 8

1.2.5 5. Ví Dụ Cụ Thể Phân Tích Đầy Đủ Các Lớp 9

1.3 Chuyên Sâu Tầng Padding (Vùng Đệm Nội Bộ & Khoảng Cách Lõi) 10

1.3.1 1. Bản Chất Của Padding: Vùng Đệm Thở Của Nội Dung 10

1.3.2 2. Hệ Thống Thuộc Tính Đơn Lẻ & Quy Tắc Viết Tắt (Shorthand TRouBLe) 11

1.3.3 3. Cơ Chế Tương Tác Giữa Padding Và Mô Thức Box-Sizing 12

1.3.4 4. Những Lưu Ý Kỹ Thuật Sống Còn Khi Sử Dụng Padding 13

1.3.5 5. Thực Nghiệm Kỹ Thuật Số: Đo Đạc Chiều Rộng & Chiều Cao Trên DevTools 14

1.4 Thuộc Tính `box-sizing` & Công Thức Tính Tổng Kích Thước Hộp 17

1.4.1 1. Mô Thức 1: `box-sizing: content-box;` (Mặc Định Chuẩn W3C) 17

1.4.2 2. Mô Thức 2: `box-sizing: border-box;` (Quy Chuẩn Vàng Hiện Đại) 18

1.4.3 3. Ví Dụ Đo Đạc Toàn Diện Chiếc Hộp Chuẩn Công Nghiệp 19

1.5 Ba Lưu Ý Sống Còn Khi Làm Việc Với CSS Box Model 20

Chương 2: Nghệ Thuật Kiểm Soát Kích Thước (Width & Height Masterclass) 21

2.1 Tổng Quan Thuộc Tính `width` & `height` Trong CSS 21

2.1.1 1. Thuộc Tính `width` (Chiều Rộng) 21

2.1.2 2. Thuộc Tính `height` (Chiều Cao) 23

2.1.3 3. Các Thuộc Tính Liên Quan và Quan Trọng 24

2.1.4 4. Khi Nào Nên Dùng auto Và Khi Nào Nên Dùng Giá Trị Cố Định? 26

2.2 Lưu Ý Chi Tiết Khi Sử Dụng `width` & `height` . . . 27
2.2.1 Giá Trị auto – Bản Chất Bất Đối Xứng Và Cơ Chế Kích Thước Tự Động . . . 27
2.2.2 Đơn Vị Tương Đối Và Đơn Vị Tuyệt Đối . . . 32
2.2.3 Kiểm Soát Giới Hạn: `min-width`, `max-width`, `min-height`, `max-height` . . . 33
2.2.4 Mối Tương Quan & Ảnh Hưởng Từ `display` Và `position` . . . 42
2.2.5 Tương Tác Giữa `overflow` Và Kích Thước `width` / `height` . . . 44
2.2.6 Lời Khuyên Quan Trọng & Thiết Lập Reset Toàn Cục Chuẩn . . . 45
2.3 Chuyên Đề Thực Chiến: Làm Chủ Thuộc Tính `max-width` (Responsive Masterclass) . . . 46
2.3.1 Khái Niệm & Ý Nghĩa Cốt Lõi Của `max-width` . . . 46
2.3.2 Sự Khác Biệt Giữa `width` Và `max-width` . . . 47
2.3.3 Các Ví Dụ Ứng Dụng Thực Tế Của `max-width` . . . 49
2.3.4 4. Kỹ Thuật Phối Hợp: `max-width` Kết Hợp Với `min-width` . . . 52
2.3.5 Thực Nghiệm Thực Tế Với Lớp `.MAX` & Đối Chiếu Chrome DevTools . . . 53
2.3.6 6. Khi Nào Dùng max-width? & Bảng Tổng Hợp Giá Trị Thường Dùng . . . 55
2.4 Chuyên Đề Thực Chiến: Làm Chủ Thuộc Tính `max-height` . . . 56
2.4.1 Bản Chất & Cơ Chế Hoạt Động Của `max-height` . . . 56
2.4.2 So Sánh Toàn Diện Giữa `height` Và `max-height` . . . 57
2.4.3 Thực Nghiệm Thực Tế: `max-height` Phụ Thuộc Vào Nội Dung . . . 58
2.4.4 Cạm Bẫy Thực Chiến: Xung Đột Giữa `height` Cố Định Và `max-height` . . . 59
2.4.5 Các Ứng Dụng Thực Tế Tiêu Biểu Của `max-height` . . . 60
2.4.6 Khi Nào Nên Dùng `max-height` & Lưu Ý Thực Chiến . . . 62

Phần I

CSS3 Nâng Cao – Mô Hình Hộp & Bố Cục Hiện Đại

CHƯƠNG 1

Chuyên Đề Toàn Diện: CSS Box Model (Mô Hình Hộp)

[Khái Niệm] Khởi Tạo Tài Liệu CSS Tập 2: Nền Tảng Mô Hình Hộp

Mọi phần tử HTML trên trang web (khung bài viết, nút bấm, hình ảnh, thanh điều hướng, ô nhập liệu) về bản chất đều được trình duyệt xử lý như một chiếc hộp hình chữ nhật (**Box**). Việc thấu hiểu tường tận cấu trúc của mô hình hộp (**CSS Box Model**) và thuật toán tính toán kích thước `box-sizing` chính là chiếc chìa khóa nền tảng số một để bạn làm chủ toàn bộ hệ thống bố cục giao diện web hiện đại.

1.1 Bản Chất Của Khái Niệm "Box" & Cấu Trúc CSS Box Model

1.1.1 1. Khái Niệm Cốt Lõi: Mọi Phần Tử Đều Là Chiếc Hộp

Trong tư duy thiết kế web của trình duyệt, **CSS Box Model (Mô hình hộp)** là cơ chế dựng hình nền tảng cho từng phần tử HTML: mỗi phần tử đều được xem như một chiếc hộp hình chữ nhật, gồm nhiều lớp đồng tâm bao quanh nội dung.

[Ghi Chú] Triết Lý Nền Tảng Của CSS Layout

Mọi phần tử HTML đều là một "box" — bất kể bạn có nhìn thấy viền của nó hay không! Từ các phần tử khối lớn (`<div>`, `<section>`), khối văn bản (`<p>`, `<h1>`), cho đến các phần tử nội dòng (`<span>`, `<a>`, `<img>`), trình duyệt đều áp dụng cùng một mô hình hộp chữ nhật để tính toán vị trí và diện tích hiển thị.

1.1.2 2. Bản Chất Hoạt Động Khi Trình Duyệt Vẽ (Render)

Khi trình duyệt tiến hành vẽ (**render**) một phần tử lên màn hình, bộ máy hiển thị (**rendering engine**) sẽ thực hiện tính toán qua 4 bước logic tuần tự:

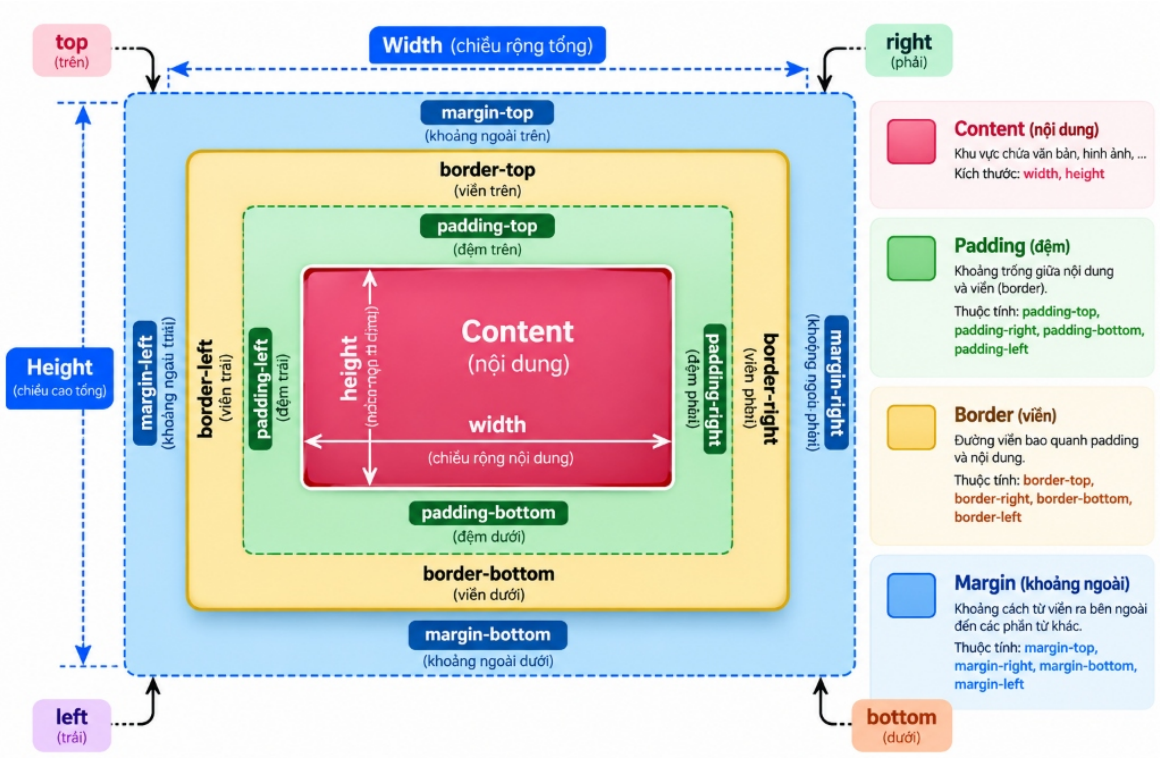
1. **Nội dung gì cần hiển thị? (Content)**: Đo lường kích thước chữ viết, hình ảnh hoặc các phần tử con

bên trong.

2. **Có khoảng đệm không? (Padding)**: Kiểm tra khoảng cách đệm từ nội dung đến đường viền.
3. **Có viền bao quanh không? (Border)**: Tính toán độ dày và kiểu dáng của đường viền bao quanh.
4. **Có khoảng cách với phần tử khác không? (Margin)**: Xác định cự ly đẩy ra xa các phần tử lân cận.

$\Longrightarrow$ **Kết luận**: Trình duyệt sẽ cộng dồn các lớp không gian này lại với nhau để định hình chính xác kích thước thực tế và tọa độ hiển thị của phần tử trên trang web.

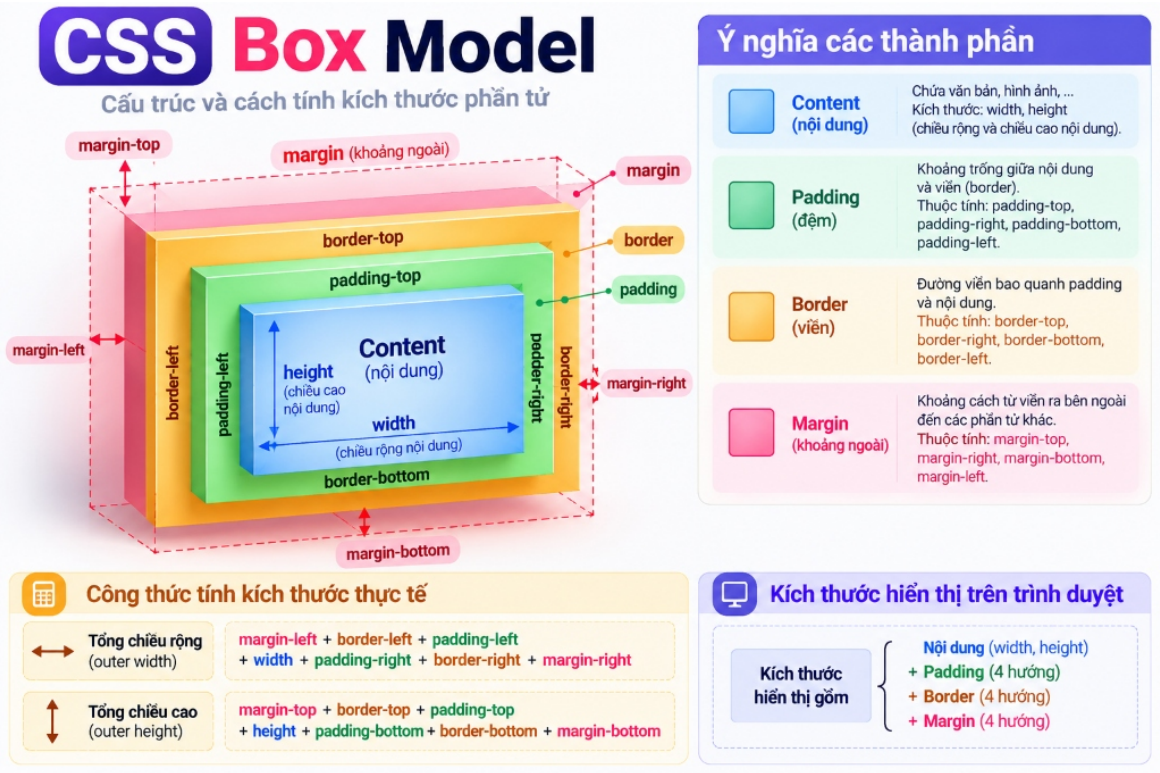
$$\textbf{Box} = \textbf{Content} + \textbf{Padding} + \textbf{Border} + \textbf{Margin} \tag{1.1}$$



Hình 1.1: Sơ Đồ 2D Trực Diện Của CSS Box Model: Phân Tách 4 Phân Tầng Cùng 4 Hướng Không Gian (Top, Right, Bottom, Left)

[Bảng] Bảng Chi Tiết 4 Thành Phần Cốt Lõi Của CSS Box Model

Thành Phần	Vị Trí Không Gian	Ý Nghĩa Chức Năng & Quy Tắc Kỹ Thuật
`content`	Lõi trong cùng	Vùng chứa nội dung thực sự (chữ viết, hình ảnh, video...). Kích thước cốt lõi được điều khiển bởi `width` và `height`.
`padding`	Đệm giữa nội dung và viền	Khoảng trống trong suốt bao quanh nội dung. Mang màu nền (`background-color`) của phần tử. **Tuyệt đối không nhận giá trị âm**.
`border`	Đường bao quanh đệm	Đường viền bao bọc toàn bộ `padding` và `content`. Có thể định dạng nét vẽ (`solid`, `dashed`), độ dày và màu sắc.
`margin`	Lớp ngoài cùng	Khoảng cách từ viền ngoài của phần tử đến các phần tử lân cận. Hoàn toàn trong suốt. **Có thể nhận giá trị âm** để kéo đè phần tử.



Hình 1.2: Mô Hình Phối Cảnh 3D Đa Tầng Của CSS Box Model & Công Thức Kích Thước Hiển Thị Thực Tế Trên Trình Duyệt

1.1.3 3. Box Model Giúp Điều Khiển Những Yếu Tố Nào?

Nắm vững Box Model cho phép bạn kiểm soát hoàn toàn 4 khía cạnh giao diện then chốt:

- **Kích thước hiển thị thực tế**: Xác định chính xác số pixel chiều rộng và chiều cao phần tử chiếm dụng.
- **Khoảng cách giữa các phần tử**: Dùng `margin` để tạo nhịp thở thoáng đãng và cự ly khoa học giữa các khối.
- **Không gian nội bộ giữa nội dung và viền**: Dùng `padding` giúp chữ viết không bị dính sát vào đường viền gây tức mắt.
- **Khả năng thích ứng khi co giãn màn hình**: Kiểm soát cách phần tử co giãn linh hoạt khi thay đổi kích thước trình duyệt.

1.1.4 4. Tại Sao Bắt Buộc Phải Hiểu Rõ CSS Box Model?

[Mẹo] 4 Lý Do Sống Còn Để Làm Chủ Box Model

1. **Kiểm soát layout chuẩn xác** 100%: Tránh hoàn toàn các lỗi kinh điển như tràn khung (**layout overflow**), vỡ dòng, lệch vị trí, hoặc kích thước bị phình to không đúng ý.
2. **Tối ưu hóa thiết kế đáp ứng (Responsive Layout)**: Tính toán chính xác tỷ lệ phần trăm và kích thước linh hoạt trên mọi độ phân giải.
3. **Nền tảng của mọi hệ thống bố cục cao cấp**: Box Model là nền tảng cốt lõi của CSS Flexbox, CSS Grid, kỹ thuật định vị (`position`), và hiệu ứng hoạt họa (`animation`).

Ví Dụ Đối Chiếu Kích Thước Giữa content-box Và border-box

```html
<div class="box">Noi dung</div>
```

```css
.box {
    width: 200px;
    padding: 10px;
    border: 5px solid black;
    margin: 20px;
}
```

- **Nếu dùng `box-sizing: content-box` (mặc định)**:

$$\text{Tổng chiều rộng thực tế} = 200\text{px} + (10\text{px} \times 2) + (5\text{px} \times 2) = \mathbf{230}\text{px}$$

- **Nếu dùng `box-sizing: border-box` (chuẩn hiện đại)**:

$$\text{Tổng chiều rộng thực tế} = \mathbf{200}\text{px} \quad \text{(đã bao trọn cả padding và border bên trong)}$$

1.2 Chuyên Sâu Tầng Content (Vùng Nội Dung Cốt Lõi)

1.2.1 1. Content Là Gì?

Content (Nội dung) là phần trung tâm nằm ở vị trí sâu nhất trong chiếc hộp CSS. Đây là nơi chứa đựng các giá trị thông tin thực tế mà người dùng nhìn thấy và tương tác: văn bản, hình ảnh, video, trường nhập dữ liệu (`input`), hoặc các phần tử con.

> **[Ghi Chú] Đặc Điểm Cốt Lõi Của Vùng Content**
>
> Theo chuẩn mặc định của W3C, khi bạn đặt thuộc tính `width` và `height` cho một phần tử, bạn đang trực tiếp ấn định kích thước cho **riêng vùng content này** (trừ khi chuyển sang mô thức `box-sizing: border-box`).

1.2.2 2. Kích Thước Của Vùng Content Dưới 2 Chế Độ Box-Sizing

- **Chế độ mặc định `box-sizing: content-box`**:

Khai Báo Với content-box

```
div {
    width: 200px;   /* Chi tinh rieng cho vung content */
    padding: 10px;
    border: 5px solid black;
}
```

$\Longrightarrow$ **Tổng chiều rộng thực tế**: $200\text{px}\ (\text{content}) + 2 \times 10\text{px}\ (\text{padding}) + 2 \times 5\text{px}\ (\text{border}) = \mathbf{230}\text{px}$.

- **Chế độ khuyến nghị `box-sizing: border-box`**:

Khai Báo Với border-box

```
div {
    box-sizing: border-box;
    width: 200px;   /* Tong kich thuoc toan bo gom content + padding + border */
    padding: 10px;
    border: 5px solid black;
}
```

$\Longrightarrow$ Con số 200px là kích thước tổng. Vùng `content` bên trong sẽ **tự động co nhỏ lại** chỉ còn:

$$200\text{px} - (10\text{px} \times 2) - (5\text{px} \times 2) = \mathbf{170}\text{px}$$ để nhường chỗ cho đệm và viền.

1.2.3 3. Content Có Thể Hiển Thị Những Gì?

Tất cả các thành phần nằm bên trong vùng Content sẽ được bảo vệ, hiển thị trọn vẹn và không bao giờ bị che khuất bởi padding hay border:

[Bảng] Các Loại Dữ Liệu Phổ Biến Hiển Thị Trong Vùng Content

Loại Nội Dung	Mã Nguồn Ví Dụ	Đặc Điểm Hiển Thị Trong Content
Văn bản (Text)	`<p>Hello CSS</p>`	Chữ viết tự động xuống dòng khi chạm mép phải của content.
Hình ảnh (Images)	`<img src="..." />`	Chiếm diện tích theo kích thước nội tại hoặc thuộc tính kích thước.
Ô nhập liệu (Form Input)	`<input type="text" />`	Các thành phần biểu mẫu tương tác người dùng.
Đa phương tiện (Media)	`<video>`, `<audio>`	Trình phát âm thanh, video nhúng.
Các phần tử con	`<div><span>Text</span></div>`	Cây phân cấp DOM lồng nhau bên trong khối cha.



Hình 1.3: Chuyên Đề Toàn Diện Về Content Trong CSS Box Model: 4 Đặc Điểm Cốt Lõi Cùng 3 Kịch Bản Hiển Thị (Văn Bản, Hình Ảnh Và Co Giãn Tự Động auto)

1.2.4 4. Cách `width` & `height` Điều Chỉnh Vùng Content

Thiết Lập Kích Thước Vùng Content

```
.box {
    width: 300px;
    height: 150px;
}
```

Vùng `content` sẽ được ấn định chính xác chiều rộng 300px và chiều cao 150px.

[Cảnh Báo] 2 Lưu Ý Quan Trọng Khi Điều Chỉnh Kích Thước Content

- **Khi không khai báo `width` hoặc `height`**: Kích thước của vùng content sẽ **tự động co giãn theo khối lượng nội dung bên trong** (hành vi mặc định `auto`).
- **Khi nội dung quá nhiều vượt quá sức chứa của Content**: Nội dung sẽ bị tràn khung (**overflow**). Lúc này bạn cần điều khiển bằng thuộc tính `overflow`:
  - `overflow: hidden;` → Cắt bỏ và ẩn toàn bộ phần nội dung bị tràn ra ngoài.
  - `overflow: scroll;` → Luôn luôn hiển thị thanh cuộn (kể cả khi nội dung chưa tràn).
  - `overflow: auto;` → Tự động sinh thanh cuộn mượt mà chỉ khi lượng nội dung thực sự vượt quá kích thước hộp.

1.2.5 5. Ví Dụ Cụ Thể Phân Tích Đầy Đủ Các Lớp

Xét ví dụ cụ thể sau để quan sát cách 4 lớp phối hợp nhịp nhàng:

Ví Dụ Phân Tách Chi Tiết 4 Lớp Box Model

```html
<div class="box">Day la phan content</div>
```

```css
.box {
    width: 200px;
    height: 100px;
    padding: 10px;
    border: 2px solid blue;
    margin: 20px;
    background-color: lightyellow;
}
```

- **Vùng Content**: Chứa chuỗi văn bản `"Day la phan content"`, kích thước rộng 200px và cao 100px.
- **Vùng Padding**: Tạo khoảng trống đệm 10px xung quanh nội dung, mang màu nền vàng nhạt (`lightyellow`).
- **Vùng Border**: Đường viền màu xanh dương (`blue`) dày 2px bao bọc lấy padding và content.
- **Vùng Margin**: Khoảng cách cách ly 20px hoàn toàn trong suốt đẩy cách xa các phần tử lân cận.

[Bảng] Bảng Tổng Kết: Mối Tương Quan Giữa Content, Kích Thước Và Box-Sizing

Thành Phần / Thuộc Tính	Mục Đích Chính Trong Bố Cục
`Content`	Chứa đựng nội dung thực tế hiển thị đến người dùng (chữ, ảnh, video...).
`width, height`	Xác định kích thước chiều rộng và chiều cao cho vùng content.
`box-sizing`	Điều chỉnh phạm vi ảnh hưởng của `width`/`height` (chỉ tính riêng content hay gộp cả border + padding).

1.3 Chuyên Sâu Tầng Padding (Vùng Đệm Nội Bộ & Khoảng Cách Lõi)

1.3.1 1. Bản Chất Của Padding: Vùng Đệm Thở Của Nội Dung

Trong mô hình hộp CSS, **Padding (Vùng đệm nội bộ)** là lớp không gian nằm giữa phần nội dung cốt lõi (`content`) và đường viền bao quanh (`border`):

Content ⟷ Padding ⟷ Border

Nhiệm vụ trọng tâm của padding là tạo ra một khoảng trống an toàn bao bọc xung quanh nội dung, giúp khối văn bản hoặc hình ảnh có một "khoảng thở" tự nhiên, tuyệt đối không bị dính sát vào đường viền gây tức mắt và khó chịu cho người đọc.

[Khái Niệm] Ẩn Dụ Kiến Trúc Về Lớp Padding

Nếu ví phần tử HTML như một gian phòng khép kín:

- **Border** chính là các bức tường nhà bao bọc xung quanh.
- **Content** là đồ nội thất đặt ở trung tâm gian phòng (bàn ghế, giường tủ, máy tính).
- **Padding** chính là khoảng cách hành lang lưu thông từ đồ nội thất đến các bức tường. Khoảng cách này càng rộng rãi, hợp lý thì căn phòng trông càng thoáng đãng, sang trọng và tiện nghi.



Hình 1.4: Tổng Quan Toàn Diện Về CSS Padding: Vị Trí Không Gian Trong Box Model, 4 Đặc Điểm Cốt Lõi, Công Thức Kích Thước Hiển Thị Thực Tế & Ứng Dụng Với Văn Bản, Hình Ảnh

1.3.2 2. Hệ Thống Thuộc Tính Đơn Lẻ & Quy Tắc Viết Tắt (Shorthand TRouBLe)

CSS cung cấp 4 thuộc tính đơn lẻ để bạn kiểm soát cự ly đệm độc lập cho từng phía của chiếc hộp:

- `padding-top`: Khoảng cách từ đỉnh nội dung đến mép viền trên.
- `padding-right`: Khoảng cách từ mép phải nội dung đến mép viền phải.
- `padding-bottom`: Khoảng cách từ đáy nội dung đến mép viền dưới.
- `padding-left`: Khoảng cách từ mép trái nội dung đến mép viền trái.

[Mẹo] Quy Tắc Vàng Ghi Nhớ Thứ Tự Viết Tắt: Thuật Nhớ TRouBLe

Để tối ưu hóa mã nguồn, lập trình viên frontend luôn sử dụng thuộc tính viết tắt `padding` theo chiều quay của kim đồng hồ bắt đầu từ vị trí 12 giờ (Đỉnh):

$$\mathbf{T} \longrightarrow \mathbf{R} \longrightarrow \mathbf{B} \longrightarrow \mathbf{L} \quad (\mathbf{T}\text{op} - \mathbf{R}\text{ight} - \mathbf{B}\text{ottom} - \mathbf{L}\text{eft} \implies \text{Thuật nhớ: } \mathbf{TRouBLe})$$

[Bảng] Ma Trận Cú Pháp Viết Tắt (Shorthand) Của Thuộc Tính Padding

Số Giá Trị	Cú Pháp Mẫu	Ý Nghĩa Phân Bổ Không Gian
1 giá trị	`padding: 10px;`	Áp dụng đồng nhất cho **cả 4 phía**: Trên, Phải, Dưới, Trái đều bằng 10px.
2 giá trị	`padding: 10px 20px;`	Giá trị 1 cho **Trên & Dưới** (10px); Giá trị 2 cho **Trái & Phải** (20px).
3 giá trị	`padding: 10px 20px 5px;`	Giá trị 1: **Trên** (10px); Giá trị 2: **Trái & Phải** (20px); Giá trị 3: **Dưới** (5px).
4 giá trị	`padding: 10px 20px 5px 15px;`	Phân bổ theo chiều kim đồng hồ (**TRouBLe**): **Trên** 10px, **Phải** 20px, **Dưới** 5px, **Trái** 15px.

Minh Họa Khai Báo Chi Tiết Từng Phía vs Viết Gọn Shorthand

```css
/* Cach 1: Khai bao chi tiet tung thuoc tinh don le */
.box-detailed {
    padding-top: 10px;
    padding-right: 20px;
    padding-bottom: 5px;
    padding-left: 15px;
}

/* Cach 2: Viet gon tuong duong 100% theo quy tac TRouBLe */
.box-shorthand {
    padding: 10px 20px 5px 15px; /* Top | Right | Bottom | Left */
}
```

1.3.3 3. Cơ Chế Tương Tác Giữa Padding Và Mô Thức Box-Sizing

Bản chất hoạt động của padding phụ thuộc trực tiếp vào mô thức tính toán kích thước mà bạn đang thiết lập cho phần tử:

- **Khi sử dụng chế độ mặc định (`box-sizing: content-box`)**: Padding sẽ **cộng dồn ra phía ngoài**, đẩy đường viền `border` ra xa và làm tăng trực tiếp kích thước hiển thị tổng thể của chiếc hộp:

$$\textbf{Layout Width} = \text{width} + \text{padding-left} + \text{padding-right} + \text{border-left} + \text{border-right}$$

- **Khi sử dụng quy chuẩn hiện đại (`box-sizing: border-box`)**: Padding **không làm tăng kích thước tổng thể** của phần tử. Thay vào đó, vùng nội dung (**Content Area**) sẽ **tự động co thắt lại**

bên trong để nhường không gian cho các lớp đệm:

$$\textbf{Content Width} = \text{width} - (\text{padding-left} + \text{padding-right}) - (\text{border-left} + \text{border-right})$$

1.3.4 4. Những Lưu Ý Kỹ Thuật Sống Còn Khi Sử Dụng Padding

[Cảnh Báo] 4 Quy Tắc Kỹ Thuật Bắt Buộc Phải Ghi Nhớ Khi Dùng Padding

1. **Tuyệt đối không nhận giá trị âm**: Khác với thuộc tính `margin` có thể nhận giá trị âm để kéo các phần tử chèn đè lên nhau, `padding` **luôn luôn phải** ≥ 0. Khai báo `padding: -10px;` là hoàn toàn sai cú pháp W3C; trình duyệt sẽ bỏ qua giá trị này hoặc báo lỗi.
2. **Quy tắc tính toán đơn vị phần trăm (%)**: Khi bạn khai báo padding theo đơn vị `%`:

   Tất cả 4 hướng (kể cả padding-top và padding-bottom) đều được tính theo CHIỀU RỘNG (width) của phần tử cha!

   Ví dụ: Khối cha có `width: 500px;`. Khối con đặt `padding: 10%;` thì cả 4 phía trên, dưới, trái, phải đều sẽ nhận cự ly đệm chính xác là:

   $$500\text{px} \times 10\% = \mathbf{50}\text{px}$$

   (Quy tắc này giúp đảm bảo tỷ lệ co giãn hình học đồng dạng và tránh vòng lặp tính toán đệ quy vô tận giữa chiều cao con và chiều cao cha).
3. **Màu nền (`background`) mặc định phủ kín trọn vẹn cả vùng Padding**: Vùng padding hoàn toàn trong suốt về mặt dữ liệu, nhưng về mặt đồ họa, màu nền (`background-color`) và hình nền (`background-image`) của phần tử sẽ **tự động trải rộng bao phủ trùm lên cả vùng Content và Padding** (chỉ dừng lại trước mép trong của viền `border`).
4. **Tăng diện tích bấm (Touch Target) cho thành phần tương tác**: Padding là vũ khí tối thượng trong thiết kế UX/UI cho các nút bấm (`<button>`), thẻ liên kết (`<a>`), và ô nhập liệu (`<input>`). Bằng cách tăng padding, ta mở rộng đáng kể diện tích tiếp xúc ngón tay trên màn hình cảm ứng mà không làm vỡ kích thước phông chữ:

Tối Ưu Diện Tích Tiếp Xúc Ngón Tay (Touch Target)

```
button {
    font-size: 14px;
    padding: 10px 20px; /* Tang dien tich bam tren man hinh cam ung */
}
```

1.3.5 5. Thực Nghiệm Kỹ Thuật Số: Đo Đạc Chiều Rộng & Chiều Cao Trên DevTools

Để thấu hiểu sâu sắc từng con số được trình duyệt tính toán ngầm bên dưới, chúng ta cùng phân tích 3 thực nghiệm kinh điển sau:

Thực nghiệm 1: Padding đơn giản với mô thức mặc định `content-box`

Thực Nghiệm 1: Padding Gây Phình To Hộp

```html
<div class="box1">Noi dung ben trong</div>
```

```css
.box1 {
    background-color: lightblue;
    padding: 20px;              /* 4 phia deu 20px */
    border: 2px solid blue;
    width: 200px;               /* Ap dung rieng cho content */
}
```



(a) Phần tử hiển thị trên trang với viền xanh và đệm 20px

(b) DevTools đo lường thực tế kích thước hiển thị: 200×61.6px

Hình 1.5: Thực Nghiệm Thực Tế Phân Tích Kích Thước Khối Lệnh `.box1` Trên Trình Duyệt Chrome

- **Phân tích chiều rộng (Width)**: Do mặc định là `content-box`, thuộc tính `width: 200px` chỉ tính riêng cho vùng chữ. Chiều rộng layout thực tế chiếm dụng là:

$$\text{Layout Width} = 200\text{px} + (20\text{px} \times 2) + (2\text{px} \times 2) = \mathbf{244}\text{px}$$

- **Phân tích chiều cao (Height) khi không khai báo `height`**: Khi ta không ấn định thuộc tính `height`, trình duyệt sẽ tự động đo lường chiều cao của dòng chữ bên trong (**content height** dựa trên `font-size` và `line-height`, ở đây xấp xỉ 17.6px đến 20px):

$$\begin{aligned}\text{Layout Height} &= \text{Content Height} + \text{padding-top} + \text{padding-bottom} + \text{border-top} + \text{border-bottom} \\ &= 17.6\text{px} + 20\text{px} + 20\text{px} + 2\text{px} + 2\text{px} = \mathbf{61.6}\text{px}\end{aligned}$$

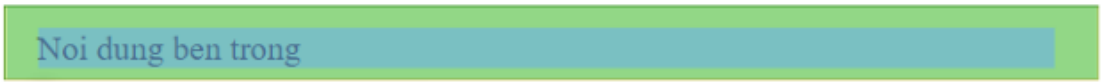
$\Longrightarrow$ Con số chính xác **200** $\times$ **61.6**px xuất hiện trên bảng phân tích DevTools (Hình 1.5b) phản ánh đúng thuật toán đo đạc này!

Thực nghiệm 2: Padding từng phía với cú pháp Shorthand 4 giá trị

Thực Nghiệm 2: Padding 4 Hướng Riêng Biệt

```
<div class="box2">Vi du padding 4 huong</div>
```

```
.box2 {
    background-color: lightgreen;
    padding: 10px 20px 5px 15px; /* Top: 10px | Right: 20px | Bottom: 5px | Left:
     15px */
    border: 1px solid green;
}
```



Hình 1.6: Thực Nghiệm Phần Tử Khối Không Khai Báo Width: Hộp Tự Động Co Giãn Tràn Kín Khung Chứa Với Lớp Đệm Bao Quanh

- **Đệm trên (`padding-top`)**: 10px cự ly với mép trên.
- **Đệm phải (`padding-right`)**: 20px khoảng cách mép phải.
- **Đệm dưới (`padding-bottom`)**: 5px khoảng cách mép dưới.
- **Đệm trái (`padding-left`)**: 15px khoảng cách mép trái.

Thực nghiệm 3: Bảo toàn kích thước tuyệt đối với `border-box`

Thực Nghiệm 3: Co Thắt Content Với border-box

```
<div class="box3">Noi dung khong lam tang kich thuoc</div>
```

```
.box3 {
    width: 200px;
    height: 100px;
    padding: 20px;
    border: 2px solid red;
    box-sizing: border-box;
    background-color: pink;
}
```

- **Kích thước hiển thị tổng thể**: Được cố định bất biến đúng chuẩn **200**px × **100**px.
- **Vùng lõi content bên trong**: Tự động co thắt đồng thời trên cả hai chiều:

$$\text{Content Width} = 200\text{px} - (20\text{px} \times 2) - (2\text{px} \times 2) = \mathbf{156}\text{px}$$
$$\text{Content Height} = 100\text{px} - (20\text{px} \times 2) - (2\text{px} \times 2) = \mathbf{56}\text{px}$$

[Bảng] Ma Trận Ứng Xử Của Chiều Rộng & Chiều Cao Khi Kết Hợp Với Padding

Kịch Bản Khai Báo	Mô Thức `content-box`	Mô Thức `border-box`
Không khai báo `width/height`	Trình duyệt tự đo theo khối lượng chữ rồi cộng thêm đệm và viền.	Vẫn tự đo theo khối lượng chữ (do không có mốc chặn cố định).
Có khai báo kích thước cố định	Kích thước ngoài **bị phình to** thêm: kích thước+padding+border.	Kích thước ngoài **được bảo toàn tuyệt đối**, phần content tự co thắt lại.

[Bảng] Bảng Tổng Kết Tra Cứu Toàn Diện Về Thuộc Tính Padding

Tiêu Chí Kỹ Thuật	Quy Chuẩn Hoạt Động & Khuyến Nghị Thực Tiễn
Vị trí không gian	Nằm giữa lõi `content` và đường viền bao quanh `border`.
Giá trị âm	**Tuyệt đối không hợp lệ** (luôn luôn ≥ 0).
Đơn vị phần trăm (%)	Cả 4 phía đều tính theo **chiều rộng (width) của phần tử cha**.
Màu nền (`background`)	Mặc định trải rộng phủ kín trọn vẹn cả vùng `content` và `padding`.
Tác động kích thước	Làm phình to hộp với `content-box`; Thu hẹp vùng lõi với `border-box`.
Ý nghĩa UX	Giúp nội dung dễ đọc, thoáng đãng; tăng diện tích bấm (**Touch Target**) cho nút và ô nhập liệu.

1.4 Thuộc Tính box-sizing & Công Thức Tính Tổng Kích Thước Hộp

Thuộc tính box-sizing quyết định trực tiếp công thức toán học mà trình duyệt dùng để tính toán kích thước hiển thị của phần tử trên màn hình. Trong CSS có hai mô thức tính toán đối lập nhau hoàn toàn:

1.4.1 1. Mô Thức 1: box-sizing: content-box; (Mặc Định Chuẩn W3C)

[Ghi Chú] Cơ Chế Phình To Kích Thước Của content-box

Đặc trưng: Thuộc tính width và height chỉ áp dụng cho riêng vùng lõi **Content Area**. Toàn bộ padding và border sẽ bị **cộng dồn ra phía ngoài**, khiến kích thước phần tử bị phình to hơn con số định sẵn trong CSS.

Hệ thống công thức tính toán chuẩn:

$$\textbf{Layout Width} = \text{width} + \text{padding-left} + \text{padding-right} + \text{border-left} + \text{border-right}$$

$$\textbf{Layout Height} = \text{height} + \text{padding-top} + \text{padding-bottom} + \text{border-top} + \text{border-bottom}$$

$$\textbf{Outer Space} = \text{Layout Width} + \text{margin-left} + \text{margin-right} \quad \text{(khi tính cả khoảng cách lề)}$$

Thực nghiệm số: Xét phần tử có khai báo `width: 200px; padding: 10px; border: 2px solid;`:

$$\text{Layout Width} = 200\text{px} + (10\text{px} \times 2) + (2\text{px} \times 2) = \mathbf{224}\text{px}$$

$$\Longrightarrow \textbf{Hộp phình to thêm 24px so với kích thước khai báo!}$$

1.4.2 2. Mô Thức 2: `box-sizing: border-box;` (Quy Chuẩn Vàng Hiện Đại)

[Mẹo] Cơ Chế Bảo Toàn Kích Thước Tuyệt Đối Của border-box

Đặc trưng: Thuộc tính `width` và `height` bao hàm trọn vẹn toàn bộ **Content + Padding + Border**. Khi tăng `padding` hoặc `border`, vùng nội dung (**Content Area**) sẽ **tự động co thắt lại bên trong** để bảo vệ tổng kích thước hộp luôn cố định đúng như khai báo.

Hệ thống công thức tính toán chuẩn:

$$\textbf{Layout Width} = \text{width} \quad \textbf{(Bất biến, không bao giờ bị phình to!)}$$

$$\textbf{Content Width} = \text{width} - (\text{padding-left} + \text{padding-right}) - (\text{border-left} + \text{border-right})$$

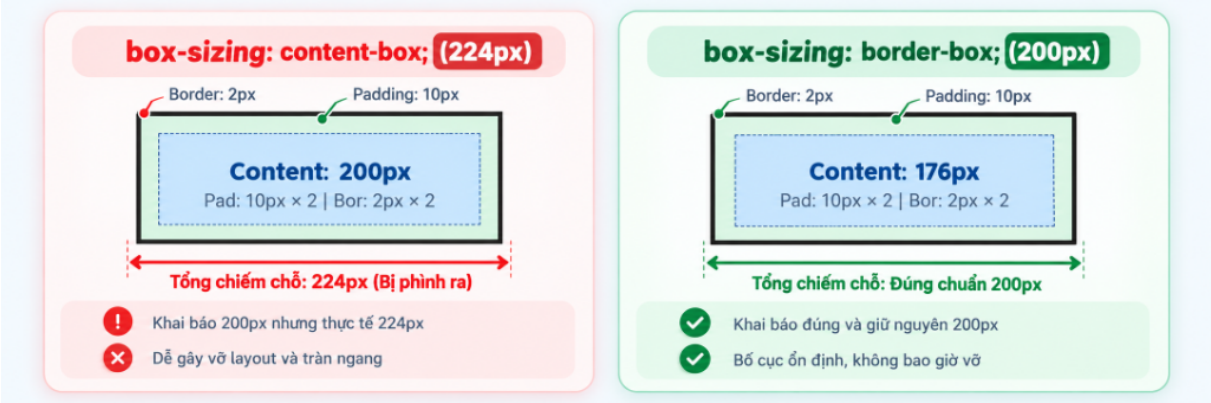
Thực nghiệm số: Xét phần tử có `width: 200px; padding: 10px; border: 2px solid; box-sizing: border-box;`:

$$\text{Content Width} = 200\text{px} - 20\text{px} - 4\text{px} = \textbf{176}\text{px}$$

$$\text{Layout Width} = \textbf{200}\text{px} \quad \textbf{(Bảo toàn chuẩn xác đúng 200px!)}$$

[Bảng] Ma Trận Đối Chiếu Trực Diện: content-box vs border-box

Tiêu Chí So Sánh	Mô Thức `content-box`	Mô Thức `border-box`
Phạm vi `width`/`height`	Chỉ áp dụng riêng cho vùng **Content**.	Bao trọn cả **Content + Padding + Border**.
Khi thêm `padding`/`border`	Hộp bị **phình to ra** ngoài dự kiến.	Vùng nội dung **tự co lại**, kích thước ngoài giữ nguyên.
Tính toán Responsive	Rất phức tạp, dễ gây tràn layout (vỡ khung).	Trực quan, dễ kiểm soát, chuẩn 100% theo thiết kế.
Ví dụ thực tế (200px + 10px + 2px)	Chiều rộng thực tế chiếm: **224**px.	Chiều rộng thực tế chiếm: đúng **200**px.
Khuyến nghị sử dụng	Ít dùng, chỉ tồn tại vì tính tương thích ngược.	**Khuyến nghị tối đa cho** 100% **dự án hiện đại**.



Hình 1.7: So Sánh Trực Quan Cơ Chế Chiếm Chỗ: `content-box` (Phình To Ra 224px) vs `border-box` (Bảo Toàn Kích Thước 200px)

1.4.3 3. Ví Dụ Đo Đạc Toàn Diện Chiếc Hộp Chuẩn Công Nghiệp

Hãy quan sát một ví dụ thực tế đo đạc đầy đủ cả 4 lớp không gian khi lập trình viên định nghĩa một thẻ `<div>`:

Đo Đạc Toàn Diện 4 Lớp Không Gian Của Hộp border-box

```
div {
    width: 200px;
    height: 120px;
    padding: 10px;
    border: 5px solid black;
    margin: 20px;
    box-sizing: border-box;
}
```

- **Chiều rộng layout (Layout Width)**: Đúng **200**px (gồm trọn viền và đệm, không bị phình to).
- **Chiều rộng vùng chữ thực tế (Content Width)**: $200\text{px} - (10\text{px} \times 2) - (5\text{px} \times 2) = \mathbf{170}\text{px}$.
- **Chiều cao vùng chữ thực tế (Content Height)**: $120\text{px} - (10\text{px} \times 2) - (5\text{px} \times 2) = \mathbf{90}\text{px}$.
- **Tổng khoảng không gian chiếm dụng trên trang (Outer Space)**: Tính thêm cự ly lề ngoài `margin: 20px;`:

$$\text{Outer Width} = 200\text{px} + (20\text{px} \times 2) = \mathbf{240}\text{px}, \quad \text{Outer Height} = 120\text{px} + (20\text{px} \times 2) = \mathbf{160}\text{px}$$

1.5 Ba Lưu Ý Sống Còn Khi Làm Việc Với CSS Box Model

[Cảnh Báo] 3 Lưu Ý Sống Còn Về Box Model Trong Lập Trình Frontend

1. **Quy tắc giá trị âm (Negative Values)**:
   - `margin` **hoàn toàn có thể nhận giá trị âm** (ví dụ: `margin-top: -20px;`). Thuộc tính này thường được dùng để kéo phần tử dịch chuyển đè lên trên khối khác hoặc mở rộng hình ảnh tràn khỏi lề container.
   - Ngược lại, `padding` **TUYỆT ĐỐI KHÔNG ĐƯỢC ÂM**! Theo chuẩn W3C, mọi giá trị âm cho padding đều bị coi là lỗi cú pháp và trình duyệt sẽ bỏ qua toàn bộ giá trị đó.
2. **Hiện tượng hợp nhất lề dọc (Margin Collapsing)**: Khi hai phần tử khối (**block-level**) đứng kề nhau theo phương dọc, lề dưới của phần tử trên và lề trên của phần tử dưới sẽ **không cộng dồn**, mà bị gộp lại và chỉ lấy giá trị lề lớn nhất!
3. **Chuẩn thiết lập Reset toàn cầu**: Để việc tính toán kích thước giao diện luôn nhất quán và không bị lỗi vỡ layout khi thêm padding hay border, quy tắc bắt buộc trong mọi dự án công nghiệp là đặt reset ngay đầu file CSS:

```
*, *::before, *::after {
    box-sizing: border-box;
}
```

CHƯƠNG 2

Nghệ Thuật Kiểm Soát Kích Thước (Width & Height Masterclass)

[Khái Niệm] Bản Chất Của Kích Thước Trong Thiết Kế Web Hiện Đại

Sau khi đã thấu hiểu chiếc hộp CSS được cấu tạo như thế nào ở Chương 1, chương này sẽ trang bị cho bạn toàn bộ nghệ thuật kiểm soát và điều phối kích thước: từ những giá trị cơ bản `width` và `height`, bản chất bất đối xứng của giá trị `auto`, bộ tứ giới hạn an toàn `min-width`, `max-width`, `min-height`, `max-height`, cho đến các chuyên đề thực chiến Responsive chuyên sâu.

2.1 Tổng Quan Thuộc Tính `width` & `height` Trong CSS

`width` và `height` là hai thuộc tính CSS cơ bản và cực kỳ quan trọng, dùng để thiết lập chiều rộng và chiều cao của một phần tử HTML. Chúng là nền tảng để kiểm soát kích thước và bố cục của mọi thứ trên trang web của bạn.

[Khái Niệm] Bản Chất Của width & height Trong Thiết Kế Web

Mọi thành phần hiển thị trên trang web (khung bài viết, nút bấm, hình ảnh, thanh điều hướng, ô nhập liệu) đều cần được định hình không gian rõ ràng. `width` và `height` đóng vai trò là chiếc thước đo cơ bản nhất để lập trình viên xác định diện tích mà một khối nội dung sẽ chiếm giữ trên mặt phẳng giao diện hai chiều.

2.1.1 1. Thuộc Tính `width` (Chiều Rộng)

Thuộc tính `width` xác định chiều rộng của vùng nội dung (**content area**) của một phần tử.

Các Giá Trị Phổ Biến Của width

1. `auto` **(Mặc định)**:
   - **Mô tả**: Trình duyệt sẽ tự động tính toán chiều rộng.
   - **Hành vi chi tiết**:

  - Đối với các phần tử khối (**block-level** như `<div>`, `<p>`, `<h1>`–`<h6>`): `width: auto;` thường khiến phần tử chiếm toàn bộ chiều rộng có sẵn của phần tử cha nó, trừ đi `margin`, `padding`, và `border`.
  - Đối với các phần tử nội dòng khối (**inline-block**) hoặc phần tử trôi nổi (**float**): `width: auto;` sẽ khiến chiều rộng co lại (**shrink-wrap**) để vừa khít với nội dung bên trong nó.
- **Khi sử dụng**: Khi bạn muốn trình duyệt tự quản lý kích thước dựa trên nội dung hoặc không gian có sẵn.

2. `length` **(Giá trị cụ thể)**:
   - **Mô tả**: Đặt một chiều rộng cố định bằng các đơn vị đo lường CSS.
   - **Hệ thống đơn vị phổ biến**:
     - `px` (**pixels**): Đơn vị cố định, không thay đổi theo môi trường hay tỷ lệ phông chữ. Ví dụ: `width: 200px;`.
     - `em`: Tương đối so với kích thước phông chữ (`font-size`) của phần tử cha (hoặc chính nó nếu có khai báo `font-size`). Ví dụ: `width: 10em;` (nếu `font-size` của phần tử cha là 16px, thì $10\text{em} = 10 \times 16\text{px} = 160\text{px}$).
     - `rem`: Tương đối so với kích thước phông chữ gốc của tài liệu (thẻ `<html>`, mặc định 16px). Ví dụ: `width: 10rem;` (tương đương $10 \times 16\text{px} = 160\text{px}$).
     - `vw` (**viewport width**): Tương đối so với chiều rộng của khung nhìn trình duyệt (**viewport**). $1\text{vw} = 1\%$ chiều rộng viewport. Ví dụ: `width: 50vw;` (chiếm chính xác 50% chiều rộng màn hình thiết bị).
     - `%` (**percentage**): Tương đối so với chiều rộng của phần tử cha trực tiếp. Ví dụ: `width: 80%;` (chiếm 80% chiều rộng vùng đệm/nội dung của cha).
   - **Khi sử dụng**: Khi bạn cần kiểm soát chính xác kích thước theo bản vẽ thiết kế (UI/UX wireframe) hoặc yêu cầu bố cục nghiêm ngặt.

3. `initial`: Đặt thuộc tính về giá trị mặc định của nó theo đặc tả W3C (`auto`).

4. `inherit`: Kế thừa giá trị chiều rộng cụ thể từ phần tử cha trực tiếp.

Minh Họa Khai Báo Các Giá Trị Của Thuộc Tính width

```css
/* 1. Gia tri mac dinh tu dong tinh toan */
.card-auto {
    width: auto;
}

/* 2. Don vi tuyet doi pixel */
.sidebar {
    width: 280px;
}

/* 3. Don vi tuong doi theo font chu */
.text-column-em {
    width: 25em;   /* Co gian theo co chu */
}

.text-column-rem {
    width: 40rem;  /* Co gian theo co chu goc html */
}

/* 4. Don vi phan tram va viewport */
.main-content {
    width: 75%;    /* Chiay 75% be rong phan tu cha */
}

.hero-banner {
    width: 100vw;  /* Phu kin 100% chieu rong man hinh */
}
```

2.1.2 2. Thuộc Tính `height` (Chiều Cao)

Thuộc tính `height` xác định chiều cao của vùng nội dung (**content area**) của một phần tử.

Các Giá Trị Phổ Biến Của height

1. `auto` **(Mặc định)**:
   - **Mô tả**: Trình duyệt sẽ tự động tính toán chiều cao dựa trên dòng chảy nội dung bên trong.
   - **Hành vi chi tiết**: Chiều cao của phần tử sẽ co lại hoặc giãn ra linh hoạt để vừa khít với toàn bộ lượng nội dung bên trong nó (chữ, hình ảnh, khối con).
   - **Khi sử dụng**: Đây là hành vi mong muốn trong hầu hết các trường hợp xây dựng trang web, giúp nội dung không bao giờ bị cắt cụt và phần tử tự động thích ứng với mọi độ dài văn bản hoặc kích thước màn hình.

2. `length` **(Giá trị cụ thể)**:
   - **Mô tả**: Đặt một chiều cao cố định bằng các đơn vị đo lường CSS.
   - **Hệ thống đơn vị**: Tương tự như `width`, bao gồm `px`, `em`, `rem`, `vh` (**viewport height**), và `%`:
     - `vh` (**viewport height**): Tương đối so với chiều cao của khung nhìn trình duyệt. 1vh = 1% chiều cao viewport. Ví dụ: `height: 100vh;` (chiếm trọn vẹn 100% chiều cao màn hình trình duyệt).
   - **Khi sử dụng**: Khi bạn cần một chiều cao chính xác, ví dụ: tạo một banner cố định, ảnh avatar, thanh điều hướng (**header bar**), hoặc các phần tử trong bố cục dạng lưới (**grid layout**) hoặc **flexbox**.
3. `initial`: Đặt thuộc tính về giá trị mặc định của nó theo quy chuẩn (`auto`).
4. `inherit`: Kế thừa giá trị chiều cao từ phần tử cha trực tiếp.

Minh Họa Khai Báo Các Giá Trị Của Thuộc Tính height

```css
/* 1. Tu dong co gian theo noi dung (Khuyen nghi cho khoi chua van ban) */
.article-body {
    height: auto;
}

/* 2. Chieu cao co dinh cho thanh tieu de (Navigation Bar) */
.navbar {
    height: 64px;
}

/* 3. Man hinh chao phu tron 100% chieu cao viewport */
.hero-fullscreen {
    height: 100vh;
}

/* 4. Khung anh vuong ti le dong */
.avatar-thumb {
    width: 80px;
    height: 80px;
}
```

2.1.3 3. Các Thuộc Tính Liên Quan và Quan Trọng

Để kiểm soát kích thước một cách linh hoạt, an toàn và chống vỡ giao diện trên nhiều thiết bị, bạn sẽ thường xuyên sử dụng kết hợp các thuộc tính sau:

[Bảng] Bảng Tra Cứu Các Thuộc Tính Kiểm Soát Kích Thước Liên Quan

Thuộc tính	Mục đích & Hành vi cốt lõi	Cú pháp mẫu
`min-width` `min-height`	Thiết lập ngưỡng kích thước tối thiểu. Phần tử không thể co nhỏ hơn ngưỡng này dù không gian bị thu hẹp.	`min-width: 300px;` `min-height: 40px;`
`max-width` `max-height`	Thiết lập ngưỡng kích thước tối đa. Phần tử không thể phình to hơn ngưỡng này dù màn hình rất rộng.	`max-width: 1200px;` `max-height: 300px;`
`box-sizing`	Quyết định thuật toán tính `width`/`height` có gộp cả `padding` và `border` vào hay không.	`box-sizing: border-box;`
`overflow`	Xử lý hành vi hiển thị khi nội dung bên trong vượt quá kích thước `width`/`height` đã định.	`overflow: auto;`

Chi Tiết Từng Thuộc Tính Bổ Trợ

1. `min-width` / `min-height`:
   - Đặt chiều rộng/chiều cao tối thiểu mà một phần tử có thể có. Phần tử sẽ không nhỏ hơn giá trị này, ngay cả khi không gian bị hạn chế.
   - Rất hữu ích trong thiết kế đáp ứng (**responsive design**) để ngăn phần tử bị quá nhỏ dẫn đến méo mó hoặc không đọc được chữ.
   - **Ví dụ**: `min-width: 300px;`.
2. `max-width` / `max-height`:
   - Đặt chiều rộng/chiều cao tối đa mà một phần tử có thể có. Phần tử sẽ không lớn hơn giá trị này, ngay cả khi có nhiều không gian trống.
   - Cũng rất hữu ích trong thiết kế đáp ứng, ví dụ: `max-width: 100%;` cho hình ảnh để chúng tự co lại linh hoạt và không bao giờ tràn ra ngoài phần tử cha.
   - **Ví dụ**: `max-width: 600px;`.
3. `box-sizing`:
   - Đây là một thuộc tính cực kỳ quan trọng ảnh hưởng trực tiếp đến cách `width` và `height` được tính toán.
   - `content-box` (**Mặc định**): `width` và `height` chỉ áp dụng cho vùng nội dung của phần tử. `padding` và `border` được thêm vào bên ngoài kích thước này, làm cho phần tử thực tế lớn hơn so với con số khai báo.
   - `border-box`: `width` và `height` bao gồm cả `padding` và `border`. Đây là mô hình hộp trực quan hơn và dễ làm việc hơn trong bố cục. Nó được khuyến nghị sử dụng trong hầu hết các dự án hiện đại.
4. `overflow`:
   - Kiểm soát những gì xảy ra khi nội dung của một phần tử tràn ra khỏi kích thước `width` và `height` đã định.

- Các giá trị phổ biến:
  - `visible` (Mặc định): Nội dung tràn hẳn ra ngoài phần tử và đè lên các thành phần xung quanh.
  - `hidden`: Cắt bỏ phần nội dung thừa vượt ra ngoài khung chứa và ẩn nó đi.
  - `scroll`: Luôn luôn hiển thị thanh cuộn (cả ngang lẫn dọc), cho phép người dùng cuộn xem phần bị tràn.
  - `auto`: Chỉ hiển thị thanh cuộn khi nội dung thực sự vượt quá kích thước khung chứa.

Ví Dụ So Sánh Kích Thước Với box-sizing: content-box vs border-box

```
/* Ap dung cho tat ca cac phan tu de thong nhat cach tinh toan */
* {
    box-sizing: border-box;
}

.box {
    width: 200px;
    padding: 10px;
    border: 2px solid black;
    /* Voi border-box, chieu rong tong cong van la dung 200px. */
    /* Voi content-box, chieu rong tong cong se la: 200 + 10*2 + 2*2 = 224px. */
}
```

2.1.4 4. Khi Nào Nên Dùng auto Và Khi Nào Nên Dùng Giá Trị Cố Định?

1. **Khi nên sử dụng `width: auto;` và `height: auto;`:**
   - **Trong hầu hết các trường hợp, đặc biệt là với `height`**: Bạn thường muốn phần tử tự động điều chỉnh chiều cao theo lượng nội dung để tránh nội dung bị cắt cụt (**clipping**) khi văn bản dài ra hoặc đổi ngôn ngữ.
   - **Khi bạn muốn phần tử khối (block) chiếm toàn bộ chiều rộng có sẵn**: Tận dụng cơ chế lấp đầy tự nhiên của trình duyệt mà không cần khai báo cứng `width: 100%`.
   - **Khi bạn muốn phần tử `inline-block` co lại vừa với nội dung**: Tạo các nút bấm (**button**), nhãn (**badge**) hay thẻ (**tag**) vừa vặn với chiều dài từ ngữ bên trong.
2. **Khi nên sử dụng `width` và `height` với giá trị cụ thể (`px`, `%`, `vw`, `vh`…)**:
   - Khi bạn cần kiểm soát kích thước chính xác cho một thiết kế cụ thể (ví dụ: kích thước logo, hình ảnh đại diện avatar, khung chứa cố định).
   - Để tạo các cột có chiều rộng nhất định (ví dụ: sidebar rộng 280px) hoặc các vùng có chiều cao tối thiểu/tối đa.
   - Khi làm việc với mô hình bố cục hiện đại như **Flexbox** hoặc **CSS Grid** để định hình các phần tử con (**flex/grid items**).

[Khái Niệm] Viên Gạch Đầu Tiên Của Bố Cục Giao Diện Web

`width` và `height` là những viên gạch đầu tiên trong việc xây dựng bố cục web. Kết hợp chúng một cách thông minh với các thuộc tính liên quan như `min-width`, `max-width`, `box-sizing`, và `overflow` sẽ giúp bạn tạo ra những giao diện linh hoạt, đáp ứng (**responsive**) và đẹp mắt trên mọi thiết bị.

2.2 Lưu Ý Chi Tiết Khi Sử Dụng `width` & `height`

Trong quá trình xây dựng giao diện thực tế, việc sử dụng `width` và `height` có thể dẫn đến nhiều lỗi hiển thị khó lường nếu không nắm chắc 5 nguyên lý cơ bản sau đây.

2.2.1 Giá Trị auto – Bản Chất Bất Đối Xứng Và Cơ Chế Kích Thước Tự Động

Trong CSS, `auto` là một từ khóa đặc biệt mang tính quyết định đến cách thức bố cục vận hành. Thiết lập `width: auto;` hoặc `height: auto;` (hoặc hoàn toàn **không khai báo** kích thước) đồng nghĩa với việc bạn chuyển giao toàn bộ quyền tính toán kích thước cho thuật toán phân bổ không gian (**Normal Flow Algorithm**) của trình duyệt.

Thuật toán này sẽ tính toán kích thước dựa trên 4 yếu tố ngữ cảnh cốt lõi:

1. **Loại phần tử**: Phần tử khối (**block-level**), phần tử nội dòng (**inline-level**), hay phần tử thay thế (**replaced element** như `<img>`).
2. **Nội dung bên trong**: Chiều dài văn bản, số lượng dòng chữ, hoặc kích thước pixel tự nhiên (**intrinsic size**) của tệp đa phương tiện.
3. **Không gian khả dụng của phần tử cha**: Chiều rộng khả dụng của khung chứa hoặc các ràng buộc bố cục nâng cao (`flex`, `grid`).
4. **Mô hình hộp và các thuộc tính kiểm soát**: Tác động trực tiếp từ `box-sizing`, `padding`, `border`, `min-*`, `max-*`.



Hình 2.1: Infographic Tổng Quan: Cơ Chế Tự Động Tính Toán Kích Thước Của `width: auto;` & `height: auto;`

[Khái Niệm] Khái Niệm Cốt Lõi: Bản Chất Bất Đối Xứng Của Giá Trị auto Giữa Hai Trục

Nhiều lập trình viên mới học thường thắc mắc: *"Tại sao `width: auto;` lại tự động bung trọn khung chứa, trong khi `height: auto;` lại co cụm ôm sát nội dung?"*
Câu trả lời nằm ở **nguyên lý dòng chảy tự nhiên của tài liệu web**:

- **Trục ngang (Inline Axis)** có giới hạn vật lý nghiêm ngặt bởi khung nhìn thiết bị (viewport). Để thuận tiện cho việc trình bày văn bản theo dòng, các phần tử khối mặc định có tính chất "thèm khát không gian ngang" (**greedy width**) — tự động bung ra chiếm 100% không gian cha để chứa được nhiều chữ nhất trên mỗi dòng.
- **Trục dọc (Block Axis)** của trang web có độ dài vô hạn thông qua thao tác cuộn chuột. Nếu phần tử cũng tự động bung 100% chiều cao cha, các khối phía sau sẽ bị đẩy rơi xuống đáy, tạo ra các khoảng trống khổng lồ vô nghĩa. Vì vậy, `height: auto;` được thiết kế mang tính chất "tiết kiệm không gian dọc" — chỉ mở rộng vừa đủ để chứa trọn lượng nội dung con bên trong.

a) Đối Với `width: auto;` & Cuộc Chạm Trán Với `width: 100%;`

Khi áp dụng `width: auto;`, hành vi sẽ phân hóa rõ rệt tùy theo chế độ hiển thị:

- **Phần tử khối (Block-level elements, ví dụ `<div>`, `<p>`, `<section>`)**: Tự động chiếm toàn bộ chiều rộng có sẵn trong phần tử cha (trừ đi margin, padding, border của chính nó).
- **Phần tử nội dòng (Inline-level elements, ví dụ `<span>`, `<a>`, `<strong>`)**: **Hoàn toàn phớt lờ** mọi khai báo `width` cố định. Kích thước bề ngang chỉ bằng đúng độ dài chuỗi ký tự hoặc hình ảnh nội dòng bên trong.

- **Phần tử nội dòng khối (`inline-block`) hoặc Phần tử trôi nổi (`float`)**: Tự động co lại ôm sát nội dung (**shrink-wrap**), chỉ mở rộng khi nội dung bên trong dài thêm.

[Ghi Chú] Giải Mã Cốt Lõi: So Sánh width: auto vs width: 100% & Tác Động Của box-sizing

Đây là điểm gây hiểu lầm và lỗi vỡ bố cục phổ biến nhất trong xây dựng giao diện CSS:

- **Khi dùng `width: auto;`**: Trình duyệt tự giải phương trình kích thước thông minh:

$$\text{Chiều Rộng Nội Dung} = \text{Chiều Rộng Cha} - (\text{Margin} + \text{Padding} + \text{Border})$$

  Nhờ cơ chế tự bù trừ này, phần tử **KHÔNG BAO GIỜ bị tràn khung cha**, ngay cả khi dự án chưa thiết lập `box-sizing: border-box;`.
- **Khi dùng `width: 100%;`**: Bạn đang ép bề ngang vùng đệm (**content area**) bằng chính xác 100% không gian cha. Nếu sau đó bạn thêm `padding: 20px;` hoặc `border: 5px;`, tổng bề ngang thực tế sẽ bằng 100% + 50px, dẫn đến việc **hộp bị phình to vỡ tràn ra ngoài khung chứa**, sinh ra thanh cuộn ngang gây lỗi giao diện nghiêm trọng!

b) Đối Với `height: auto;` & Cơ Chế Khi Không Khai Báo height

Khi lập trình viên **không khai báo `height`**, trình duyệt sẽ tự động mặc định áp dụng `height: auto;`. Cơ chế vận hành cụ thể gồm 4 kịch bản liên kết chặt chẽ:

1. **Kịch bản 1: Nội dung rỗng & không có padding**: Chiều cao phần tử sụp đổ về đúng 0px. Khối hộp hoàn toàn vô hình trên trang dù bạn có tô màu nền bắt mắt.
2. **Kịch bản 2: Có nội dung văn bản/con**: Chiều cao co giãn mềm dẻo và tự thích ứng với số lượng dòng chữ (1 dòng thì mỏng, nhiều dòng thì đáy hộp tự động đẩy sâu xuống).
3. **Kịch bản 3: Tác động của padding khi không có height**: Chiều cao luôn được bảo toàn tối thiểu bằng padding-top + padding-bottom. Một khối rỗng chữ nhưng có `padding: 20px;` vẫn sẽ hiển thị một khối màu chữ nhật dày 40px rõ ràng.
4. **Kịch bản 4: Khi nào chiều cao KHÔNG bằng chiều cao nội dung?**:
   - **Khi có `height` cố định**: Bị đóng băng kích thước; nếu nội dung dài hơn sẽ bị tràn (**overflow**).
   - **Khi có `min-height`**: Đóng vai trò **chốt chặn sàn** an toàn, đảm bảo giao diện không bị co rúm xẹp lép khi dữ liệu chưa tải về kịp từ máy chủ.
   - **Khi có `max-height`**: Đóng vai trò **chốt chặn trần**, ngăn phần tử kéo dài vô tận làm hỏng cấu trúc trang; khi vượt ngưỡng sẽ kích hoạt thanh cuộn dọc nếu đi kèm `overflow-y: auto;`.



Hình 2.2: Thực Nghiệm Bố Cục: Không Khai Báo vs Khai Báo `width: auto; height: auto;` Cho Cùng Kết Quả Trong Khung Chứa 600px



Hình 2.3: Đối Chiếu Trực Quan: `width: auto; height: auto;` Bảo Toàn Kích Thước Gốc vs Kích Thước Cố Định Làm Méo Ảnh

[Cảnh Báo] Cạm Bẫy Kinh Điển: Thuộc Tính height: 100% Bị Liệt Trên Cha auto!

Một phần tử con khai báo `height: 100%;` sẽ **chỉ hoạt động nếu phần tử cha có chiều cao tường minh bằng con số cụ thể** (ví dụ: `height: 400px;` hoặc `height: 100vh;`).
Nếu phần tử cha đang để mặc định `height: auto;`, trình duyệt rơi vào vòng lặp vô định: *Con đòi* 100% *của Cha, nhưng Cha lại đợi Con nở ra mới biết mình cao bao nhiêu*. Kết quả: Trình duyệt hủy bỏ lệnh `height: 100%;` và con tự teo lại theo nội dung của nó!

Ví Dụ Sửa Cạm Bẫy height: 100%

```
<!-- TRUONG HOP LOI: .child khong the dat 100% vi .parent la auto -->
<div class="parent" style="height: auto;">
    <div class="child" style="height: 100%; background: lightcoral;">
        Con bi teo lai vi cha khong co chieu cao xac dinh!
    </div>
</div>

<!-- GIAI PHAP CHUAN: Xac dinh chieu cao cho cha hoac dung viewport unit -->
<div class="parent-fixed" style="height: 300px;">
    <div class="child" style="height: 100%; background: lightgreen;">
        Con nhan du 100</div></div>
```

[Mẹo] Góc Đọc Thêm: Vì Sao auto Trên Thẻ img Lại Khác Với Thẻ div?

Nhiều bạn đọc tinh ý sẽ nhận ra: Tại sao ở Hình 2.1, thẻ `<div>` có `width: auto;` lại bung 100% màn hình; nhưng ở Hình 2.3, thẻ `<img>` có `width: auto;` lại giữ đúng 1200px của ảnh gốc mà không bung theo khung chứa?
Bí mật nằm ở sự khác biệt giữa hai nhóm phần tử:

- Thẻ `<div>`, `<p>` là **phần tử không thay thế** (**non-replaced elements**): Bản thân chúng là các khung chứa cấu trúc rỗng, kích thước bề ngang hoàn toàn do thuật toán dòng chảy văn bản quy định.
- Thẻ `<img>`, `<video>` là **phần tử thay thế** (**replaced elements**): Nội dung của chúng được tải từ tệp ngoài. Bản thân tệp ảnh đã sở hữu **kích thước tự nhiên** (**intrinsic width/height**) và **tỷ lệ khung hình** (**intrinsic aspect ratio**). Khi đặt `width: auto; height: auto;`, trình duyệt diễn giải là: *"Hãy hiển thị trung thực đúng kích thước gốc của bức ảnh"*.
- Vì vậy, nếu tệp ảnh gốc rộng tới 1200px, nó sẽ đâm toang khung chứa 600px! Đây chính là lý do vì sao công thức responsive bất hủ `max-width: 100%; height: auto;` ra đời (sẽ được phân tích chi tiết tại Mục 2.2.3).

[Bảng] Ma Trận Tổng Hợp: So Sánh Bản Chất Các Thuộc Tính Đo Lường Cơ Bản

Thuộc Tính	Giá Trị Mặc Định	Cơ Chế Tính Toán Của Trình Duyệt	Lời Khuyên Kiến Trúc Frontend
`width: auto`	Mặc định thẻ HTML	Block: chiếm trọn 100% ngang cha. Inline: vừa khít chữ. Replaced: lấy ảnh gốc.	Dùng cho cấu trúc layout linh hoạt, tự thích ứng với padding/border.
`width: 100%`	Tự định nghĩa	Ép vùng đệm nội dung bằng đúng 100% cha. Dễ tràn khung nếu thiếu border-box.	Chỉ dùng khi cần ép rộng, luôn đi kèm `box-sizing: border-box`.
`height: auto`	Mặc định thẻ HTML	Co dãn theo nội dung con. Bằng 0 nếu rỗng và không có padding.	Luôn là lựa chọn hàng đầu cho văn bản để tránh lỗi tràn chữ khi responsive.
`min-height`	`0` (hoặc auto)	Tạo ngưỡng sàn tối thiểu, cho phép nở cao hơn nếu nội dung vượt mức.	Dùng cho Card sản phẩm, Hero section để chống thụt xẹp giao diện.
`max-height`	`none`	Tạo ngưỡng trần tối đa, thường kết hợp `overflow-y: auto`.	Dùng cho Modal pop-up, danh sách bình luận, khung chat box.

2.2.2 Đơn Vị Tương Đối Và Đơn Vị Tuyệt Đối

a) Đơn Vị Tuyệt Đối (Absolute Units): `px`, `pt`, `cm`, `in`...

- Cố định, không thay đổi theo kích thước màn hình thiết bị hay kích cỡ phông chữ của người dùng.
- **Lưu ý thực chiến**: Việc lạm dụng và sử dụng `px` quá nhiều cho `width` và `height` của các container lớn có thể làm cho trang web trở nên cứng nhắc, kém linh hoạt và cực kỳ khó khăn khi chuyển đổi giao diện đáp ứng (**responsive**) trên màn hình điện thoại di động hoặc máy tính bảng.

b) Đơn Vị Tương Đối (Relative Units): `%`, `em`, `rem`, `vw`, `vh`...

- **`%` (Phần trăm)**: Luôn luôn tương đối so với kích thước của phần tử cha trực tiếp. Nếu phần tử cha không có kích thước rõ ràng hoặc đang để `height: auto;`, đơn vị `%` có thể không hoạt động như mong đợi.
- `em`: Tương đối so với `font-size` của chính phần tử đó (hoặc của phần tử cha nếu `font-size` không được định nghĩa trên chính nó). Rất hữu ích cho các thành phần UI có kích thước tỷ lệ thuận với văn bản (ví dụ: nút bấm có `padding` và `width` giãn nở nhịp nhàng theo cỡ chữ).
- `rem`: Tương đối so với `font-size` của phần tử gốc `<html>`. Giúp duy trì tỷ lệ kích thước hài hòa và

đồng nhất trên toàn bộ hệ thống trang web.

- `vw` (**Viewport Width**) / `vh` (**Viewport Height**): Tương đối so với kích thước thực của cửa sổ trình duyệt (**viewport**). 1vw là 1% chiều rộng viewport, 1vh là 1% chiều cao viewport. Đây là lựa chọn tuyệt vời cho các phần tử cần chiếm một tỷ lệ cố định của màn hình thiết bị, ví dụ: `height: 100vh;` để tạo một **Hero Section** tràn đầy trọn vẹn màn hình mở đầu.
- **Lưu ý**: Các đơn vị tương đối giúp tạo ra thiết kế linh hoạt, co giãn mượt mà và đáp ứng tốt hơn trên mọi thiết bị hiện đại.

2.2.3 Kiểm Soát Giới Hạn: `min-width`, `max-width`, `min-height`, `max-height`

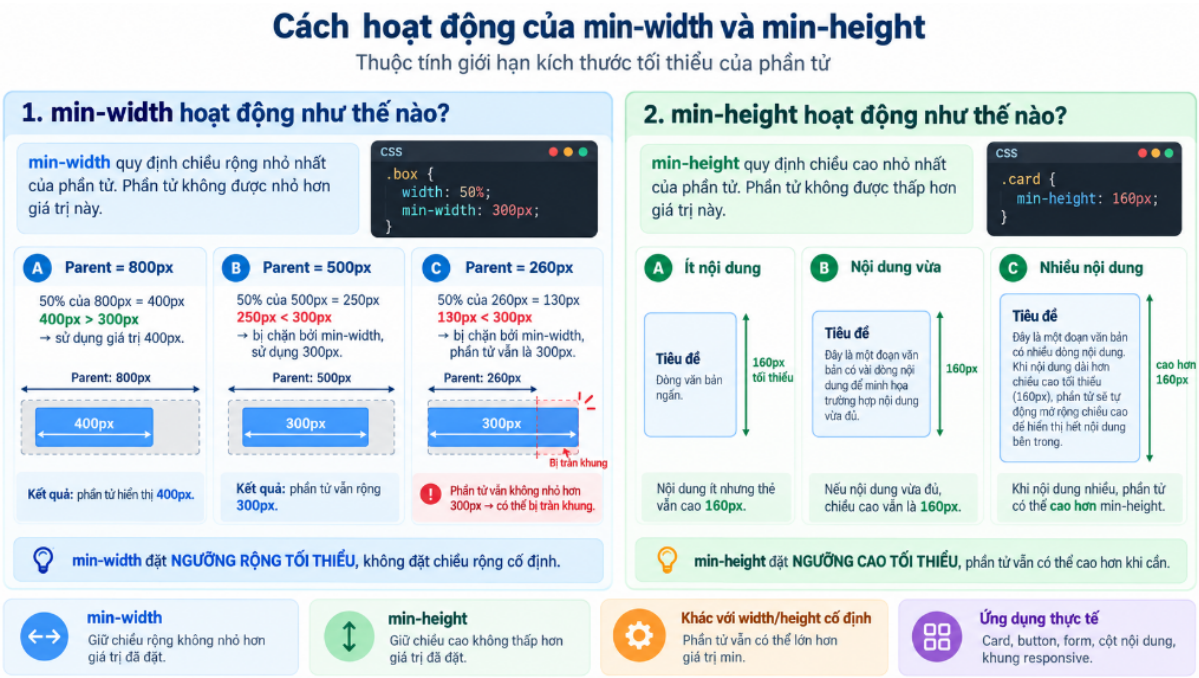
Trong kỹ thuật thiết kế giao diện đáp ứng hiện đại (**Responsive Web Design**), việc chỉ sử dụng kích thước cơ bản (`width`, `height`) là hoàn toàn không đủ. W3C cung cấp nhóm 4 thuộc tính ranh giới cực kỳ quan trọng, chia làm hai nhóm đối xứng: **Nhóm Chốt Chặn Sàn** (`min-*`) và **Nhóm Chốt Chặn Trần** (`max-*`).

a) Nhóm Chốt Chặn Sàn: `min-width` Và `min-height`

`min-width` và `min-height` là hai thuộc tính CSS giúp thiết lập **ngưỡng kích thước sàn tối thiểu** cho phần tử:

- `min-width`: Định nghĩa chiều rộng tối thiểu của phần tử. Nếu nội dung bên trong yêu cầu không gian lớn hơn giá trị này, phần tử sẽ tự động mở rộng theo nội dung.
- `min-height`: Định nghĩa chiều cao tối thiểu của phần tử. Nếu nội dung bên trong cần chiều cao lớn hơn, phần tử sẽ tự động mở rộng tương ứng.

> **[Cảnh Báo] Nguyên Lý Ưu Tiên Tối Cao Của Nhóm min-***
>
> Nếu một phần tử được khai báo giá trị kích thước cơ sở `width` hoặc `height` **nhỏ hơn** `min-width` hoặc `min-height`, thì trình duyệt sẽ **luôn ưu tiên áp dụng giá trị min-***, còn giá trị cơ sở nhỏ hơn kia hoàn toàn bị vô hiệu hóa!



Hình 2.4: Infographic Toàn Cảnh: Cơ Chế Giới Hạn Kích Thước Tối Thiểu & Các Trạng Thái Thực Nghiệm Của `min-width` & `min-height`

1. Cách Hoạt Động Cụ Thể Của `min-width`: Cơ chế của `min-width` được thể hiện qua 3 trường hợp kinh điển:

- **Trường hợp 1 (`min-width` lớn hơn `width` cơ sở)**:

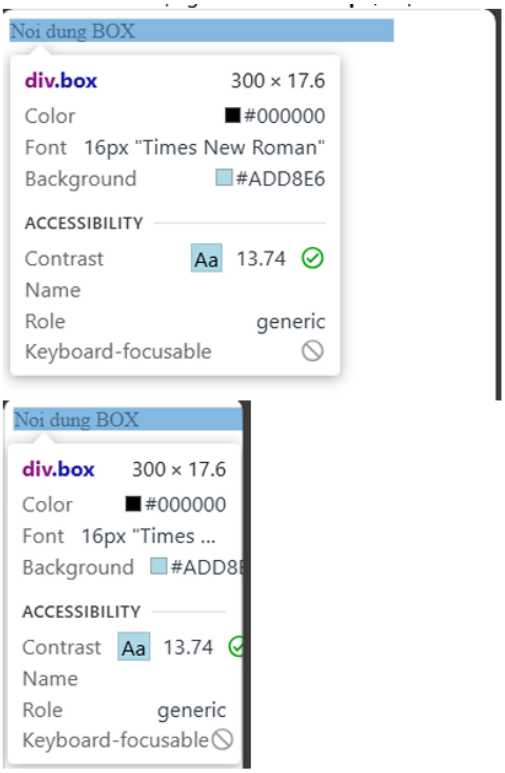
min-width Đè Bẹp width Nhỏ Hơn

```
.box {
    background: lightblue;
    width: 100px;
    min-width: 300px;
}
<div class="box">Noi dung BOX</div>
```

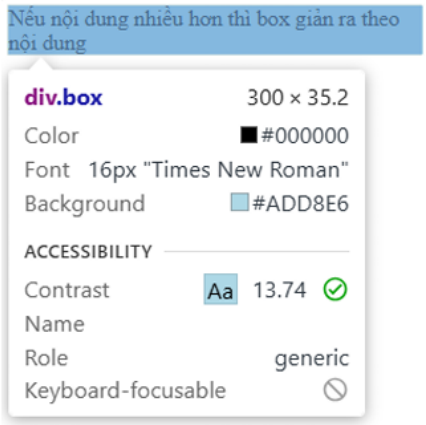
Kết quả: Phần tử luôn có ít nhất 300px chiều rộng, dù `width` chỉ đặt 100px. Nếu nội dung dài hơn 300px, phần tử tự động dãn ra theo nội dung.
Kiểm chứng thực tế trên Chrome DevTools:

– Cửa sổ Inspect hiển thị chính xác kích thước thực tế là **300** × **17.6**px (Hình 2.5).

– Khi tăng độ dài văn bản (*"Nếu nội dung nhiều hơn thì box giản ra theo nội dung"*), văn bản rớt xuống 2 dòng, chiều cao tự nở ra **35.2**px (Hình 2.6), còn bề ngang vẫn giữ nguyên 300px.



Hình 2.5: DevTools: `min-width: 300px` Đè Bẹp `width: 100px`



Hình 2.6: DevTools: Khi Thêm Chữ, Chiều Cao Tự Nở Ra Đạt 35.2px

- **Trường hợp 2 (`min-width` nhỏ hơn `width` cơ sở)**:

width Cơ Sở Lớn Hơn Chiếm Quyền Ưu Tiên

```
.box {
    background: lightgreen;
    width: 400px;     /* Lon hon min-width */
    min-width: 200px;
}
```

Kết quả: Vì `width: 400px;` lớn hơn ngưỡng sàn `min-width: 200px;`, nên kích thước thực tế hiển thị sẽ là 400px.

- **Trường hợp 3 (`min-width` kết hợp `max-width`)**:

Giới Hạn Trong Khoảng Bề Ngang An Toàn

```
.box {
    background: orange;
    min-width: 200px;
    max-width: 400px;
}
```

Kết quả: Phần tử không bao giờ nhỏ hơn 200px và không bao giờ lớn hơn 400px. Nếu nội dung ít,

phần tử rộng 200px. Nếu nội dung dài, phần tử dừng lại ở 400px.

- **Thực nghiệm kết hợp với đơn vị phần trăm (%)** (Đối chiếu Hình 2.4):
  Với khai báo `.box { width: 50%; min-width: 300px; }`:
  - **Khung cha** 800**px**: $50\% = 400\text{px} > 300\text{px} \rightarrow$ Hiển thị 400px.
  - **Khung cha** 500**px**: $50\% = 250\text{px} < 300\text{px} \rightarrow$ Bị chặn lại ở 300px.
  - **Khung cha** 260**px**: $50\% = 130\text{px} < 300\text{px} \rightarrow$ Phần tử vẫn giữ 300px $\rightarrow$ **Bị tràn ra ngoài khung cha** (minh họa viền đỏ trong Infographic).

2. Cách Hoạt Động Cụ Thể Của `min-height`: `min-height` đảm bảo chiều cao tối thiểu cho khối hộp, trải qua 3 trường hợp:

- **Trường hợp 1 (`min-height` lớn hơn `height` cơ sở)**:

min-height Đè Bẹp height Nhỏ Hơn

```
.box {
    background: lightcoral;
    height: 100px;      /* Nho hon min-height */
    min-height: 300px;
}
```

Kết quả: Chiều cao phần tử luôn đạt ít nhất 300px, hoàn toàn bỏ qua giá trị 100px.

- **Trường hợp 2 (`min-height` nhỏ hơn `height` cơ sở)**:

height Cơ Sở Lớn Hơn Chiếm Quyền Ưu Tiên

```
.box {
    background: violet;
    height: 400px;      /* Lon hon min-height */
    min-height: 200px;
}
```

Kết quả: Vì `height: 400px;` lớn hơn ngưỡng tối thiểu 200px, nên phần tử hiển thị chính xác 400px.

- **Trường hợp 3 (`min-height` kết hợp `max-height`)**:

Dải Chiều Cao An Toàn Kèm Thanh Cuộn

```
.box {
    background: lightgray;
    min-height: 200px;
    max-height: 400px;
    overflow: auto;
}
```

Kết quả: Chiều cao phần tử luôn dao động trong khoảng $[200\text{px}, 400\text{px}]$. Nếu lượng văn bản vượt

trần 400px, thanh cuộn tự động xuất hiện nhờ `overflow: auto;`.

- **Ba cấp độ co giãn theo nội dung** (Đối chiếu Hình 2.4 với thẻ `.card { min-height: 160px; }`):
  - **Ít nội dung**: Thẻ vẫn cao tối thiểu đúng 160px, không bao giờ bị teo nhỏ xẹp lép.
  - **Nội dung vừa**: Chiều cao vẫn duy trì 160px chuẩn mực.
  - **Nhiều nội dung**: Thẻ tự động nở cao vượt mốc 160px để bao trọn văn bản con mà không cắt chữ.

[Bảng] Bảng So Sánh Toàn Diện 6 Thuộc Tính Kích Thước Trong CSS

Thuộc Tính	Mô Tả Chức Năng	Hành Vi Chi Tiết Khi Nội Dung Co Giãn
`min-width`	Giữ chiều rộng tối thiểu	Nếu nội dung cần rộng hơn, phần tử tự động mở rộng theo nội dung.
`min-height`	Giữ chiều cao tối thiểu	Nếu nội dung cần cao hơn, phần tử tự động giãn cao theo nội dung.
`max-width`	Giữ chiều rộng tối đa	Nếu nội dung rộng hơn, phần tử bị giới hạn (tránh tràn khung).
`max-height`	Giữ chiều cao tối đa	Nếu nội dung cao hơn, phần tử bị khóa trần (sinh cuộn nếu có overflow).
`width: auto;`	Chiều rộng tự động	Block: bung 100% khung cha. Inline: vừa khít chữ. Replaced: ảnh gốc.
`height: auto;`	Chiều cao tự động	Ôm khít theo nội dung; bằng 0 nếu rỗng và không có padding.

[Ghi Chú] Tóm Tắt Ứng Dụng Thực Chiến Của min-width & min-height

- **Ứng dụng min-width**: Giữ layout không bị co rúm trên mobile; đảm bảo nút bấm (`button`), ô nhập liệu (`input`), thanh trượt luôn có diện tích chạm ngón tay (**touch target** $\geq$ 44**px**) theo chuẩn tiếp cận (a11y).
- **Ứng dụng min-height**: Đảm bảo khối giao diện luôn có chiều cao cố định tối thiểu (Hero banner, Card sản phẩm), giữ cấu trúc dạng lưới đồng đều ngay cả khi dữ liệu chưa tải xong hoặc có ít chữ.
- **Nguyên tắc ghi nhớ**: Khi nội dung nhỏ hơn $\rightarrow$ Kích thước giữ nguyên ở mức `min-*`; Khi nội dung lớn hơn $\rightarrow$ Phần tử tự động mở rộng theo nội dung (trừ khi bị chặn bởi `max-*`).

[Mẹo] Góc Đọc Thêm & Mẹo Phỏng Vấn: Thuật Toán W3C (min > max) & Cạm Bẫy Flexbox min-width: 0

1. **Xung đột hiếm gặp giữa min và max (Chuẩn W3C Sizing Algorithm)**: Nếu bạn vô tình đặt `min-width: 300px;` nhưng lại khai báo `max-width: 200px;`, thuộc tính nào sẽ thắng? Theo đặc tả chuẩn W3C: **`min-*` luôn luôn chiến thắng tuyệt đối!** Trình duyệt sẽ tự động cưỡng chế nâng `max-width` lên bằng mức `min-width` (300px) để đảm bảo ngưỡng an toàn sàn không bao giờ bị phá vỡ.
2. **Cạm bẫy Flexbox & Grid (`min-width: auto;`)**: Mặc định trong CSS Flexbox và CSS Grid, các phần tử con trực tiếp (**Flex/Grid items**) sở hữu giá trị mặc định là `min-width: auto;` chứ không phải là 0. Điều này khiến phần tử con từ chối co lại nhỏ hơn nội dung văn bản bên trong, dẫn đến việc tràn khung và vỡ layout khi chứa chuỗi văn bản dài không ngắt dòng (**nowrap**) hoặc bảng dữ liệu rộng.
3. **Mẹo cứu nguy thần thánh**: Luôn thiết lập khai báo `min-width: 0;` cho phần tử con trực tiếp của Flexbox/Grid khi bạn muốn nó tự do co giãn theo đúng tỷ lệ `flex-shrink` hoặc kích hoạt thành công hiệu ứng cắt chữ ba chấm (`text-overflow: ellipsis;`).

b) Nhóm Chốt Chặn Trần: `max-width` Và `max-height`

Nếu nhóm chốt chặn sàn bảo vệ giao diện không bị teo nhỏ, thì nhóm chốt chặn trần đóng vai trò bảo vệ cấu trúc trang không bao giờ bị phá vỡ bởi nội dung quá lớn:

- **`max-width: 100%;` (Quy chuẩn vàng tối thượng cho hình ảnh)**: Giúp hình ảnh không bao giờ tràn ra khỏi vùng chứa của nó, tự động co lại một cách linh hoạt khi màn hình thu hẹp hơn kích thước gốc của ảnh:

Bộ Quy Tắc Ảnh Responsive Chuẩn Công Nghiệp

```
img {
    max-width: 100%;
    height: auto;
    display: block;
}
```

Thuộc tính `height: auto;` đi kèm đảm bảo tỷ lệ khung hình (**aspect ratio**) của bức ảnh được bảo toàn nguyên vẹn, không bao giờ bị méo mó.

- **`max-height` (Khóa trần chiều cao kết hợp cuộn nội bộ)**: Cực kỳ hữu ích khi bạn muốn giới hạn chiều cao tối đa của một khối nội dung để tránh nó kéo dài trang web quá mức (ví dụ: khung bình luận, danh sách thông báo, hộp thoại điều khoản). Thường kết hợp với `overflow-y: auto;` để sinh ra thanh cuộn dọc mượt mà:

Khung Giới Hạn Chiều Cao Có Cuộn Nội Bộ

```
.modal-body, .comment-list {
    max-height: 350px;
    overflow-y: auto;
}
```

[Mẹo] Chiến Lược Thực Chiến Khi Sử Dụng min-width & max-width

- **Khóa biên dưới an toàn (`min-width`)**: Thiết lập cho các nút bấm hành động (button, input, badge) để tránh bị co rúm trên thiết bị di động màn hình hẹp (< 360px).
- **Khóa trần thị giác (`max-width`)**: Thiết lập cho các khối văn bản đọc (`max-width: 700px` đến `800px`) nhằm giữ độ dài dòng đọc ở mức lý tưởng (khoảng 60–75 ký tự/dòng theo chuẩn Ergonomics).
- **Cặp bài trùng Responsive**: Luôn ưu tiên công thức `width: 100%; max-width: 1200px; margin: 0 auto;` để tạo khối khung chứa trung tâm hoàn hảo cho mọi trang web.

c) Sự Kết Hợp Giữa max-width, max-height Và width, height: Quy Luật Hoạt Động & Trạng Thái Vô Hiệu Hóa

Trong thực tế xây dựng giao diện đáp ứng (**Responsive**), nhóm thuộc tính đo lường cơ bản (`width`, `height`) và nhóm giới hạn trần (`max-width`, `max-height`) thường xuyên được kết hợp với nhau. Tuy nhiên, nếu lập trình viên không hiểu rõ thứ tự ưu tiên của thuật toán CSS, các thuộc tính này có thể **triệt tiêu lẫn nhau** hoặc gây ra lỗi méo mó giao diện nghiêm trọng.

1. Bản Chất Đối Lập Giữa Kích Thước Cơ Sở Và Giới Hạn Trần:

- `width` / `height`: Xác định kích thước cơ sở mà phần tử muốn đạt được (có thể là cố định bằng `px` hoặc co giãn theo `auto`/`%`).
- `max-width` / `max-height`: Thiết lập **ranh giới trần tuyệt đối**. Phần tử có thể co nhỏ hơn ngưỡng này, nhưng **tuyệt đối không bao giờ được phép vượt quá**.

2. Hai Trạng Thái Tương Tác Sống Còn:

1. **Trạng thái 1: Khi `max-width` hoặc `max-height` hoạt động đúng hiệu lực**
   Bạn **không đặt kích thước cố định**, mà để kích thước tự do co giãn theo nội dung bằng cách sử dụng `width: auto;` hoặc `height: auto;`:

Thiết Lập Co Giãn Linh Hoạt Đúng Chuẩn

```
.box {
    max-width: 500px; /* Chieu rong toi da khong vuot qua 500px */
    width: auto;       /* Chieu rong co gian tu nhien theo noi dung */
}
```

Cơ chế hoạt động:

- Nếu nội dung bên trong ngắn (< 500px) $\rightarrow$ Khối hộp tự động thu nhỏ vừa khít nội dung.
- Nếu nội dung bên trong rất dài (> 500px) $\rightarrow$ Khối hộp tự động dừng lại ở ngưỡng 500px, không bao giờ vượt quá.

2. **Trạng thái 2: Khi `max-width` hoặc `max-height` bị vô hiệu hóa (Bị khóa cứng)**
   Nếu bạn vô tình hoặc cố ý thiết lập `width` hoặc `height` bằng một con số cố định nhỏ hơn giới hạn trần:

Cạm Bẫy Vo Hieu Hoa max-width

```
.box {
    width: 400px;     /* Chieu rong bi khoa cung o 400px */
    max-width: 500px; /* HOAN TOAN BI VO HIEU HOA! */
}
```

Hậu quả: Khối hộp luôn luôn có bề ngang đúng 400px, bất kể nội dung bên trong nhiều hay ít. Khai báo `max-width: 500px;` trở nên hoàn toàn vô nghĩa và bị lãng phí vì `width` đã khóa chết kích thước ở mức thấp hơn.
Quy tắc ưu tiên của CSS: Thuộc tính `max-width` chỉ phát huy tác dụng ghi đè khi kích thước tính toán của `width` có xu hướng **vượt quá** giá trị trần (ví dụ: `width: 600px; max-width: 500px;` $\rightarrow$ kết quả hiển thị thực tế sẽ bị ép xuống 500px).

3. Thực Chiến 8 Trường Hợp Phối Hợp Kích Thước Hình Ảnh: Hình ảnh (`<img>`) là ví dụ điển hình và trực quan nhất minh họa sự tương tác giữa các thuộc tính này. Xét một bức ảnh có kích thước gốc 1200×800px (tỷ lệ chuẩn $3:2$) đặt trong một khung chứa rộng 600px:



Hình 2.7: Infographic Toàn Cảnh: Sự Kết Hợp Giữa `max-width`, `max-height` Và `width`, `height` Qua 8 Trường Hợp Thực Chiến

1. **Trường hợp 1 (Không khai báo)**: Ảnh lấy kích thước gốc 1200px, tràn toang khung chứa 600px và gây thanh cuộn ngang khó chịu.
2. **Trường hợp 2 (Dùng `width: auto; height: auto;`)**: Cho kết quả tương tự Trường hợp 1 (vì `auto` trên ảnh là giữ nguyên kích thước pixel gốc của tệp tin).
3. **Trường hợp 3 (Dùng `max-width: 100%; height: auto;`)**: **Chuẩn mực vàng Responsive**: Ảnh tự động thu nhỏ vừa khít chiều rộng khung chứa (600px), chiều cao tự điều chỉnh theo đúng tỷ lệ 3 : 2 thành 400px.
4. **Trường hợp 4 (Dùng `width: 300px; height: auto;`)**: Chiều rộng bị cố định ở 300px, chiều cao tự động tính toán theo tỷ lệ thành 200px.
5. **Trường hợp 5 (Dùng `max-width: 400px; max-height: 300px;` kèm `auto`)**: Giới hạn kích thước theo cả 2 chiều; ảnh tự động dừng ở 400 × 267px, không vượt quá bất kỳ chiều nào và giữ trọn vẹn tỷ lệ.
6. **Trường hợp 6 (Đặt cả `width` và `height` cố định** 300 × 300**px)**: **Lỗi méo ảnh nghiêm trọng**! Tỷ lệ gốc 3 : 2 bị bóp méo thành hình vuông 1 : 1.
7. **Trường hợp 7 (Dùng `max-width: 300px; max-height: 300px;` kèm `auto`)**: Thu nhỏ linh hoạt để không vượt quá 300px ở bất kỳ chiều nào, kết quả đạt 300 × 200px hoàn hảo.
8. **Trường hợp 8 (Dùng `width: 100%; max-height: 300px;`)**: Ảnh luôn bung đầy 100% khung chứa (600px), nhưng chiều cao bị chặn đứng ở trần 300px.

[Bảng] Bảng So Sánh Nhanh Tác Dụng Của 4 Thuộc Tính Kích Thước

Thuộc Tính	Tác Dụng Bố Cục	Ví Dụ Giá Trị	Hành Vi Thực Tế
`width`	Đặt chiều rộng cơ sở	`300px`, `100%`, `auto`	Cố định hoặc co giãn; có thể làm méo ảnh nếu đi cùng height cố định.
`height`	Đặt chiều cao cơ sở	`200px`, `auto`	Tự điều chỉnh khi đặt auto (và có width phù hợp).
`max-width`	Thiết lập trần chiều rộng tối đa	`500px`, `100%`	Phần tử không bao giờ nở rộng vượt quá giá trị này.
`max-height`	Thiết lập trần chiều cao tối đa	`300px`	Khóa trần chiều cao, ngăn nội dung kéo dài trang vô tận.
`auto`	Ủy quyền tính toán cho trình duyệt	`auto`	Với ảnh: giữ nguyên tỷ lệ gốc khi chỉ đặt 1 chiều cố định.

[Ghi Chú] Quy Tắc Vàng Khi Kết Hợp max-width / max-height

- **Nguyên tắc 1**: Luôn sử dụng `max-width` hoặc `max-height` khi bạn muốn phần tử **co giãn mềm dẻo theo nội dung** nhưng không được phép phá vỡ giới hạn khung chứa.
- **Nguyên tắc 2**: **Tuyệt đối không đặt đồng thời `width` và `height` cố định** cho các phần tử hình ảnh/video, vì hành động này sẽ phá hủy tỷ lệ khung hình tự nhiên và làm méo mó nội dung.
- **Nguyên tắc 3**: Muốn `max-width` hoặc `max-height` phát huy toàn bộ sức mạnh responsive, hãy luôn kết hợp chúng với giá trị cơ sở `width: auto;` hoặc `height: auto;`.

2.2.4 Mối Tương Quan & Ảnh Hưởng Từ `display` Và `position`

Hành vi của `width` và `height` phụ thuộc mật thiết vào cách phần tử được định kiểu hiển thị (**display mode**) và định vị (**positioning context**).



1. Chế độ hiển thị nội dòng (`display: inline;`):

- Các phần tử nội dòng (như `<a>`, `<span>`, `<strong>`, `<em>`) sẽ **hoàn toàn không nhận** các thuộc tính `width`, `height`, `margin-top`, `margin-bottom`, `padding-top`, và `padding-bottom` theo cách thông thường.
- Chiều rộng và chiều cao của chúng được xác định hoàn toàn bởi dung lượng chữ bên trong.

2. Chế độ khối nội dòng (`display: inline-block;`):

- Các phần tử này thừa hưởng khả năng thiết lập `width` và `height` độc lập như một khối (**block**), nhưng đồng thời vẫn nằm thẳng hàng trên cùng một dòng với các phần tử nội dòng khác.

3. Chế độ định vị thoát luồng (`position: absolute;` / `position: fixed;`):

- Khi một phần tử được định vị tuyệt đối hoặc cố định, nó sẽ **bị rút hoàn toàn khỏi dòng chảy tài liệu thông thường** (**normal document flow**).
- `width` và `height` của chúng có thể được thiết lập độc lập hoặc có thể dùng đồng thời cặp thuộc tính vị trí `top: 0; bottom: 0;` và `left: 0; right: 0;` để tự động kéo giãn phủ kín không gian.
- **Ví dụ điển hình**: Khai báo `width: 100%; height: 100%; position: absolute; top: 0; left: 0;` sẽ làm cho lớp phủ (**overlay**) lấp đầy toàn bộ phần tử cha gần nhất có khai báo `position` khác `static`.

2.2.5 Tương Tác Giữa overflow Và Kích Thước width / height

Khi bạn thiết lập `width` hoặc `height` cố định cho một phần tử và khối lượng nội dung bên trong lại lớn hơn không gian kích thước đó, thuộc tính `overflow` sẽ quyết định cách thức trình duyệt xử lý phần nội dung bị tràn:



Hình 2.8: Mô Phỏng Trực Quan 4 Cơ Chế Xử Lý Của Thuộc Tính `overflow` (visible, hidden, scroll, auto)

- **`overflow: visible;`** (**Mặc định**): Nội dung bị tràn sẽ không bị cắt bỏ mà tràn ra ngoài đường biên của phần tử, đè lên các khối giao diện khác ở phía dưới.
- **`overflow: hidden;`**: Phần nội dung bị tràn ra ngoài kích thước hộp sẽ bị cắt bỏ thẳng tay và hoàn toàn ẩn đi, người dùng không thể xem được.
- **`overflow: scroll;`**: Luôn luôn hiển thị thanh cuộn (dù nội dung có bị tràn hay không), cho phép người dùng cuộn xem toàn bộ dữ liệu.
- **`overflow: auto;`**: Trình duyệt thông minh chỉ hiển thị thanh cuộn khi nội dung thực sự bị tràn vượt quá giới hạn của hộp chứa.

2.2.6 Lời Khuyên Quan Trọng & Thiết Lập Reset Toàn Cục Chuẩn

[Mẹo] Quy Tắc Vàng Khởi Đầu Cho Mọi Dự Án Web Hiện Đại

Trong hầu hết mọi dự án web hiện đại (từ website cá nhân cho tới các ứng dụng doanh nghiệp quy mô lớn), bạn nên **luôn luôn bắt đầu bằng việc thiết lập `box-sizing: border-box;` cho tất cả các phần tử** trong tệp tin CSS reset hoặc stylesheet toàn cục:

```
*, *::before, *::after {
    box-sizing: border-box;
}
```

Lợi ích thực tế: Thiết lập này sẽ giúp bạn quản lý kích thước các phần tử một cách trực quan và dễ dàng hơn rất nhiều. Mọi con số `width` và `height` bạn đặt ra sẽ luôn đúng tuyệt đối với diện tích chiếm chỗ trên màn hình, loại bỏ vĩnh viễn những lỗi tính toán sai lệch kích thước gây vỡ bố cục khi thêm `padding` và `border`.

2.3 Chuyên Đề Thực Chiến: Làm Chủ Thuộc Tính `max-width` (Responsive Masterclass)

Trong thiết kế web hiện đại, **Responsive Web Design** (thiết kế thích ứng đa thiết bị) là tiêu chuẩn bắt buộc. Nếu như `width` đại diện cho sự áp đặt kích thước cố định, thì `max-width` chính là chìa khóa mở ra sự linh hoạt và giải phóng trang web khỏi nguy cơ vỡ khung bố cục trên các màn hình nhỏ.

2.3.1 Khái Niệm & Ý Nghĩa Cốt Lõi Của `max-width`

Thuộc tính `max-width` dùng để **giới hạn chiều rộng tối đa** mà một phần tử HTML có thể đạt tới. Nếu nội dung hoặc kích thước màn hình hiển thị nhỏ hơn giá trị `max-width`, phần tử sẽ tự động co lại để vừa khít với không gian có sẵn.

Cú Pháp Thiết Lập max-width Cơ Bản

```
.element {
    max-width: 500px;
}
```

[Khái Niệm] Cơ Chế Hoạt Động 2 Trạng Thái Của max-width

Khi thiết lập `max-width: 500px;`, trình duyệt sẽ tuân thủ nghiêm ngặt 2 nguyên tắc sau:

1. **Nếu phần tử rộng hơn** 500**px**: Nó sẽ bị chặn lại ở ngưỡng trần tối đa đúng bằng 500px, không bao giờ vượt qua.
2. **Nếu phần tử nhỏ hơn** 500**px**: Nó vẫn giữ nguyên kích thước tự nhiên của mình mà không bị ép buộc phải phình to ra 500px.

Ý Nghĩa Then Chốt Trong Thiết Kế UI/UX

- **Giới hạn chiều rộng tối đa của phần tử**: Giữ cho các khối nội dung không bị bè quá rộng trên màn hình Desktop lớn hoặc màn hình Ultra-Wide, duy trì độ dài dòng đọc lý tưởng theo công thái học thị giác.
- **Tăng cường tính linh hoạt**: Giúp giao diện hiển thị xuất sắc, mượt mà trên mọi kích cỡ màn hình từ điện thoại bỏ túi, máy tính bảng cho đến màn hình tivi lớn.
- **Bộ đôi hoàn hảo với `width: 100%`**: Đây là công thức phối hợp kinh điển nhất trong kỹ nghệ dựng layout responsive hiện đại.



Hình 2.9: Infographic Toàn Diện: Khái Niệm, Cơ Chế Hiển Thị Đa Thiết Bị & Cú Pháp max-width

2.3.2 Sự Khác Biệt Giữa `width` Và `max-width`

Sự nhầm lẫn giữa `width` và `max-width` là một trong những nguyên nhân hàng đầu gây ra lỗi thanh cuộn ngang (**Horizontal Scrollbar**) trên các thiết bị di động.

[Bảng] Bảng So Sánh Cơ Chế Hoạt Động: width: 500px vs max-width: 500px

Thuộc tính	Cách thức hoạt động của trình duyệt
`width: 500px;`	Phần tử **luôn luôn bị khóa cứng** ở chiều rộng đúng 500px bất kể kích thước màn hình thiết bị hay phần tử cha lớn hay nhỏ.
`max-width: 500px;`	Phần tử **có thể nhỏ hơn** 500**px** khi không gian bị thu hẹp, nhưng **tuyệt đối không bao giờ lớn hơn** 500**px**.

Ví Dụ Mã Nguồn So Sánh Trực Tiếp

Ví Dụ So Sánh Hai Khối: .box1 (width) vs .box2 (max-width)

```
/* Khoi 1: Dung width co dinh */
.box1 {
    width: 500px;
    background: #e53935; /* Do */
    color: white;
    padding: 12px;
}

/* Khoi 2: Dung max-width linh hoat */
.box2 {
    max-width: 500px;
    background: #1e88e5; /* Xanh duong */
    color: white;
    padding: 12px;
}
```

[Cảnh Báo] Hành Vi Hiển Thị Khi Cửa Sổ Trình Duyệt Nhỏ Hơn 500px (Ví dụ Mobile 375px)

- **Đối với `.box1` (`width: 500px;`)**: Phần tử **vẫn giữ nguyên** 500**px**, vượt quá kích thước 375px của màn hình. Kết quả: **Xuất hiện thanh cuộn ngang**, nội dung bị tràn ra ngoài rìa màn hình, gây vỡ hoàn toàn layout website!
- **Đối với `.box2` (`max-width: 500px;`)**: Phần tử **tự động co lại bằng đúng** 375**px** để vừa vặn hoàn hảo với màn hình điện thoại. Giao diện mượt mà, không sinh thanh cuộn ngang, bố cục được bảo toàn nguyên vẹn.



Hình 2.10: Đối Chiếu Trực Quan Trên Thiết Bị Di Động: `width: 500px` (Gây Tràn 125px) vs `max-width: 500px` (Tự Co Giãn Vừa Vặn)

2.3.3 Các Ví Dụ Ứng Dụng Thực Tế Của `max-width`

Ứng Dụng 1: Giới Hạn Chiều Rộng Tối Đa Cho Hình Ảnh (Responsive Images)

Đây là đoạn mã quan trọng hàng đầu trong mọi stylesheet CSS hiện đại để bảo đảm hình ảnh hiển thị hoàn hảo:

Quy Tắc Hình Ảnh Đáp Ứng Chuẩn Công Nghiệp

```
img {
    max-width: 100%;
    height: auto;
    display: block;
}
```

[Khái Niệm] Tác Dụng Kỹ Thuật

- **Không bao giờ vượt quá kích thước gốc**: Bức ảnh giữ nguyên độ nét, không bị kéo vỡ hạt (**pixelated**) khi container quá lớn.
- **Tự động co nhỏ**: Khi kích thước màn hình nhỏ hơn chiều rộng thực của ảnh, `max-width: 100%;` ép ảnh co nhỏ lại đúng bằng chiều rộng phần tử cha.
- `height: auto;`: Đảm bảo tỷ lệ khung hình (**aspect ratio**) luôn được khóa chuẩn, ảnh không bao giờ bị méo mó hình học.



Hình 2.11: Ví Dụ Thực Tế Ảnh Trong CSS: So Sánh 3 Trường Hợp (Không Khai Báo, Dùng auto Và Dùng max-width: 100%; height: auto;)

Ứng Dụng 2: Tạo Khung Chứa Nội Dung Trung Tâm (Responsive Container)

Trong thiết kế giao diện website, các lập trình viên thường dùng lớp `.container` để bao bọc toàn bộ nội dung của trang:

Mẫu Responsive Container Kinh Điển

```
.container {
    max-width: 1200px;
    width: 90%;
    margin: 0 auto;
}
```

[Bảng] Cơ Chế Thích Ứng Của Lớp .container Trên Các Môi Trường Màn Hình

Môi trường hiển thị	Hành vi hiển thị thực tế của .container
Màn hình lớn (Desktop / 4K Width $>$ 1333px)	`max-width: 1200px;` kích hoạt. Trang web được giữ cố định ở bề rộng tối đa 1200px, căn giữa màn hình nhờ `margin: 0 auto;`, không bị kéo dãn quá đà.
Màn hình nhỏ hơn (Laptop / Tablet / Mobile)	`width: 90%;` chiếm ưu thế. Khung chứa tự động chiếm trọn 90% chiều rộng màn hình, để lại 5% khoảng đệm ở mỗi bên lề trái/phải cực kỳ thanh lịch.



Hình 2.12: Mô Phỏng Thực Tế Lớp `.container`: Cố Định 1200px Trên Desktop & Co Giãn 90% Trên Mobile

Ứng Dụng 3: Khung Đọc Văn Bản Công Thái Học (Blog & Báo Điện Tử)

Khung Chứa Nội Dung Đọc Tiêu Chuẩn 800px

```
.article-container {
    width: 100%;
    max-width: 800px;
    margin: 0 auto;
    padding: 20px;
}
```

2.3.4 4. Kỹ Thuật Phối Hợp: max-width Kết Hợp Với min-width

Khi cần kiểm soát kích thước trong một dải biên độ an toàn tuyệt đối, việc kết hợp đồng thời `min-width` và `max-width` là kỹ thuật tối thượng.

[Bảng] Ý Nghĩa Cặp Thuộc Tính Giới Hạn Biên Độ

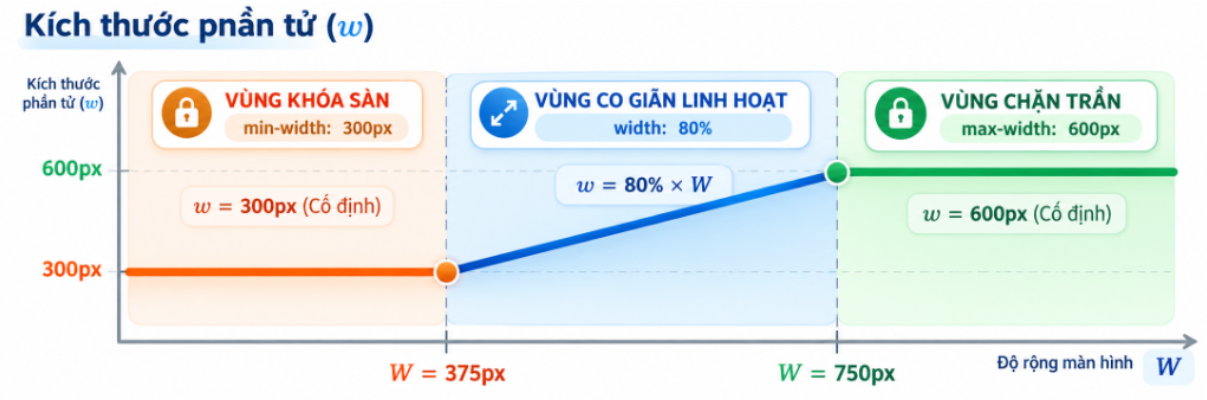
Thuộc tính	Ý nghĩa & Ngưỡng giới hạn
`max-width`	Giới hạn **ngưỡng trần tối đa** của phần tử (không thể to hơn).
`min-width`	Giới hạn **ngưỡng sàn tối thiểu** của phần tử (không thể nhỏ hơn).

Ví Dụ Phối Hợp 3 Cấp Độ Kích Thước Hoàn Hảo

```
.box {
    min-width: 300px;
    max-width: 600px;
    width: 80%;
}
```

[Khái Niệm] Cơ Chế Phân Bổ Kích Thước 3 Kịch Bản Theo Độ Rộng Màn Hình (W)

1. **Khi màn hình rất hẹp ($W < 375$px)**: $80\% \times W < 300$px, thuộc tính `min-width: 300px;` sẽ can thiệp để giữ kích thước tối thiểu là 300px, bảo đảm các nút bấm hay văn bản bên trong không bị dồn nén đến mức mất chữ.
2. **Khi màn hình nằm trong khoảng lý tưởng ($375\text{px} \leq W \leq 750$px)**: Khối co giãn mượt mà đúng bằng 80% độ rộng màn hình ($80\% \times W$), tạo cảm giác thoáng đãng và tự nhiên.
3. **Khi màn hình rất rộng ($W > 750$px)**: $80\% \times W > 600$px, thuộc tính `max-width: 600px;` sẽ chặn đứng kích thước tại ngưỡng 600px, ngăn chặn phần tử bị phình to bất thường.



Hình 2.13: Đồ Thị Trực Quan: Cơ Chế Clamping Ba Cấp Độ Kết Hợp `min-width`, `width` Và `max-width`

2.3.5 Thực Nghiệm Thực Tế Với Lớp .MAX & Đối Chiếu Chrome DevTools

Để kiểm chứng bản chất tính toán của trình duyệt, chúng ta cùng phân tích một thực nghiệm cụ thể với lớp CSS `.MAX`:

Mã Nguồn Thực Nghiệm Lớp .MAX

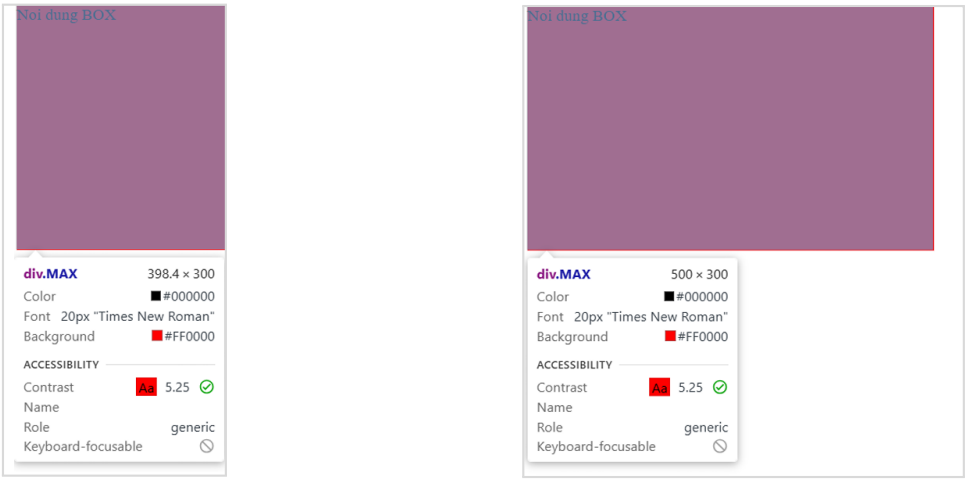
```
.MAX {
    background: red;
    max-width: 500px;
    height: 300px;
    font-size: 20px;
}
```

Phân Tích Cách Hoạt Động Cốt Lõi

1. `max-width: 500px;`: Phần tử có chiều rộng tối đa là 500px. Nếu trình duyệt nhỏ hơn 500px, phần tử co lại theo kích thước của nó; nếu trình duyệt lớn hơn 500px, phần tử không vượt quá 500px.
2. `height: 300px;`: Chiều cao của phần tử luôn là 300px, không thay đổi dù viewport rộng hay hẹp.
3. `font-size: 20px;`: Cỡ chữ của nội dung văn bản bên trong được thiết lập cố định 20px.

Điều Gì Sẽ Xảy Ra Khi Viewport Nhỏ Hơn 500px?

- **Nếu màn hình thu hẹp còn khoảng** 400**px**: Khối `.MAX` sẽ tự động co lại bằng khoảng 400px (cụ thể trên thực nghiệm Chrome DevTools đo được chính xác là **398**.**4**px × **300**px do trừ viền mép cửa sổ).
- **Nếu màn hình mở rộng ra** 600**px hoặc hơn**: Khối `.MAX` vẫn chỉ rộng đúng **500**px × **300**px, không hề tăng thêm.



(a) Viewport hẹp (< 500px): Co về 398.4px × 300px (b) Viewport rộng (> 500px): Khóa tại 500px × 300px

Hình 2.14: Hình Ảnh Chụp Thực Tế Chrome DevTools: Đối Chiếu Kích Thước Đo Lường Của Lớp .MAX

Cải Tiến Hoàn Hảo Cho Responsive

Nếu bạn muốn phần tử khối co giãn mượt mà hơn và luôn bám dính 100% không gian khi màn hình nhỏ, hãy kết hợp thêm thuộc tính `width: 100%;`:

Mẫu Thiết Kế .MAX Hoàn Thiện Tối Ưu Cho Responsive

```
.MAX {
    background: red;
    width: 100%;          /* Chiay 100% khi man hinh nho hon 500px */
    max-width: 500px;     /* Chan lai khong cho lon hon 500px tren man hinh lon */
    height: 300px;
    font-size: 20px;
}
```

2.3.6 6. Khi Nào Dùng max-width? & Bảng Tổng Hợp Giá Trị Thường Dùng

[Bảng] Bảng Tra Cứu Các Giá Trị Thường Dùng Của Thuộc Tính max-width

Giá trị	Ý nghĩa & Ngữ cảnh sử dụng
`px`	Giới hạn chiều rộng tối đa theo đơn vị pixel tuyệt đối (ví dụ: `max-width: 800px;`). Rất phổ biến cho card, form đăng nhập, container.
`%`	Giới hạn theo tỷ lệ phần trăm so với phần tử cha (ví dụ: `max-width: 100%;`). Tiêu chuẩn vàng để chống tràn cho ảnh và video.
`em, rem`	Giới hạn kích thước theo độ lớn phông chữ. Cực kỳ tối ưu cho các cột đọc báo để giữ độ dài dòng từ 65 đến 75 ký tự.
`vw`	Giới hạn theo tỷ lệ phần trăm chiều rộng của màn hình (**viewport**).
`none`	Giá trị mặc định của CSS. Phần tử không có bất kỳ giới hạn chiều rộng tối đa nào.

Khi Nào Bắt Buộc Phải Dùng max-width?

1. **Khi muốn giới hạn phần tử không quá lớn nhưng vẫn tự co lại nếu cần**: Tránh làm mất cân đối thị giác trên các màn hình máy tính để bàn có độ phân giải cao.
2. **Khi thiết kế giao diện responsive trên nhiều kích thước màn hình**: Đảm bảo trải nghiệm đồng nhất trên cả Desktop, Tablet và Mobile.
3. **Khi muốn nội dung tự động thu nhỏ trên màn hình nhỏ mà không bị vỡ**: Loại bỏ vĩnh viễn nguy cơ sinh thanh cuộn ngang gây ức chế cho người dùng.

[Mẹo] Lưu Ý Vàng Khi Sử Dụng Thuộc Tính max-width

- **Luôn kết hợp với `width: 100%;`**: Để phần tử tự do co giãn linh hoạt khi không gian nhỏ hơn giá trị `max-width`.
- **Không làm phần tử nhỏ hơn**: Giá trị `max-width` chỉ giới hạn kích thước trần lớn nhất, không ép phần tử phải co lại nếu nội dung tự nhiên của nó vốn đã nhỏ.
- **Kết hợp với `margin: 0 auto;`**: Để đưa khối giao diện vào chính giữa màn hình một cách cân đối và chuyên nghiệp.
- **Đối tượng áp dụng thường xuyên**: Khung chứa nội dung (`.container`), hình ảnh (`<img>`), biểu mẫu nhập liệu (`<form>`), thẻ bài viết (`.card`), và hộp thoại pop-up (`.modal`).

2.4 Chuyên Đề Thực Chiến: Làm Chủ Thuộc Tính max-height

Trong thiết kế giao diện web hiện đại, việc kiểm soát chiều cao của phần tử luôn tiềm ẩn nhiều thách thức hơn so với chiều rộng. Nội dung văn bản có thể dài ngắn bất định tùy thuộc vào dữ liệu động hoặc ngôn ngữ người dùng. Thuộc tính `max-height` chính là công cụ đắc lực giúp lập trình viên vừa giới hạn được kích thước trần an toàn, vừa bảo toàn tính co giãn linh hoạt tự nhiên theo nội dung.

2.4.1 Bản Chất & Cơ Chế Hoạt Động Của max-height

[Khái Niệm] Định Nghĩa Chuẩn: Thuộc Tính max-height

Thuộc tính `max-height` trong CSS dùng để **giới hạn chiều cao tối đa** mà một phần tử có thể đạt được:

- **Nếu nội dung ít hơn giới hạn**: Phần tử sẽ tự động co lại đúng bằng chiều cao tự nhiên của nội dung (với điều kiện `height: auto;`).
- **Nếu nội dung vượt quá giới hạn**: Chiều cao phần tử bị chặn cứng tại giá trị `max-height`. Phần nội dung tràn ra ngoài sẽ được xử lý tùy theo thuộc tính `overflow` đi kèm (cắt bỏ, tràn đè hoặc sinh thanh cuộn).

Cú Pháp Khai Báo & Đơn Vị

Cú Pháp Cơ Bản Thuộc Tính max-height

```
selector {
    max-height: value;
}
```

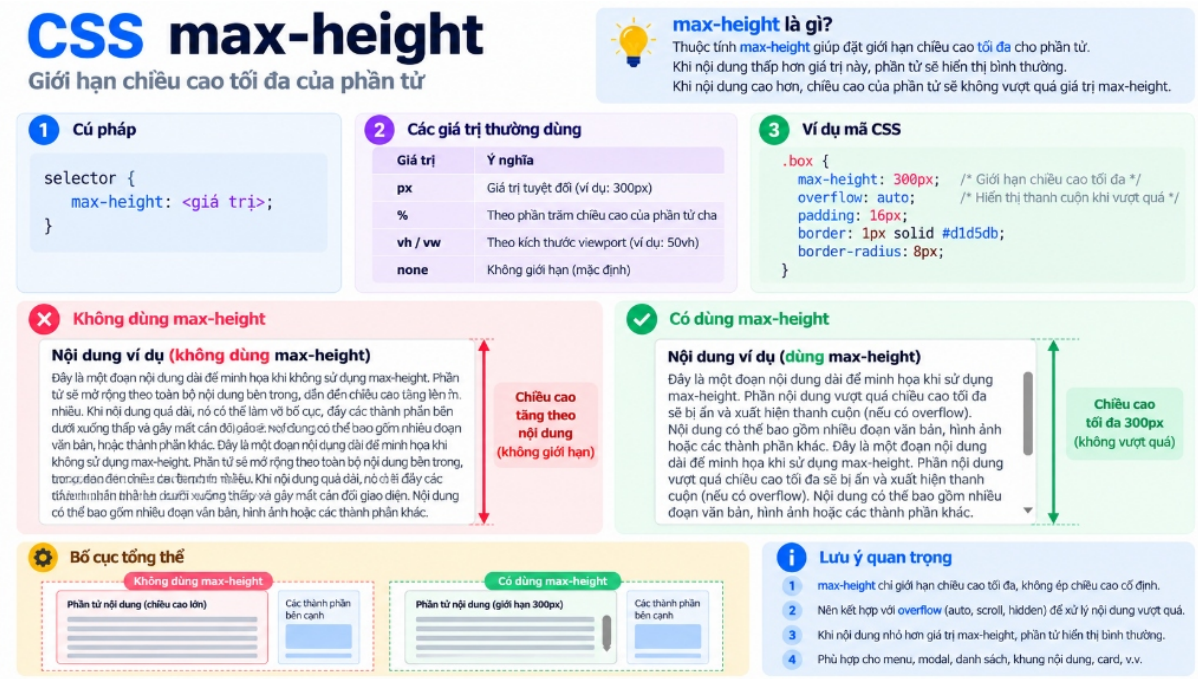
Trong đó, `value` có thể nhận các giá trị:

- **Đơn vị đo lường**: `px`, `em`, `rem`, `vh`, `vw`, `%`.
- **Từ khóa đặc biệt**: `none` (mặc định – không giới hạn chiều cao tối đa).

Ví Dụ Cơ Bản Thiết Lập max-height

```
.box {
    background: #e74c3c;
    max-height: 300px;
    width: 500px;
}
```

- Nếu khối `.box` chứa lượng nội dung ngắn có chiều cao dưới 300px, nó sẽ tự co lại ôm sát nội dung.
- Nếu khối chứa lượng nội dung đồ sộ vượt quá 300px, phần tử sẽ dừng lại ở đúng ngưỡng 300px.



Hình 2.15: Infographic Tổng Quan Thuộc Tính CSS `max-height`: Cú Pháp, Bảng Giá Trị, Mã Mẫu & Đối Chiếu Bố Cục

2.4.2 So Sánh Toàn Diện Giữa `height` Và `max-height`

Sự khác biệt căn bản giữa hai thuộc tính này nằm ở **tính linh hoạt đối với dung lượng nội dung**:

[Bảng] Bảng So Sánh Chi Tiết: height vs max-height

Tiêu chí	height	max-height
Cơ chế cốt lõi	Khóa cứng chiều cao của phần tử tại một kích thước cố định.	Thiết lập ngưỡng trần tối đa; phần tử vẫn tự do co giãn nhỏ hơn.
Khi nội dung ít	Phần tử **vẫn giữ nguyên** kích thước cố định, tạo ra khoảng trống thừa thãi lãng phí.	Phần tử **tự co lại** vừa vặn theo nội dung, tiết kiệm không gian giao diện.
Khi nội dung nhiều	Phần tử bị tràn (**overflow**) ra ngoài hoặc bị che khuất nội dung.	Phần tử đạt ngưỡng chặn trần, dễ dàng kết hợp `overflow: auto;` để cuộn.
Mức độ linh hoạt	Kém linh hoạt, dễ gây vỡ bố cục trên màn hình nhỏ.	Cực kỳ linh hoạt, chuẩn mực vàng cho thiết kế đáp ứng (**Responsive**).

Mã Nguồn So Sánh .box1 (height) vs .box2 (max-height)

```
/* Hop 1: Luon luon cao dung 300px bat ke noi dung chi co 1 dong chu */
.box1 {
    height: 300px;
    background: #3498db;
}

/* Hop 2: Co gian linh hoat theo chu, toi da chi dat 300px */
.box2 {
    max-height: 300px;
    height: auto;
    background: #2ecc71;
}
```

2.4.3 Thực Nghiệm Thực Tế: max-height Phụ Thuộc Vào Nội Dung

Khi sử dụng `max-height`, phần tử sẽ co giãn nhịp nhàng theo lượng chữ bên trong. Hãy cùng đối chiếu hai trường hợp thực tế dưới đây:



(a) Trường hợp 1: Nội dung ít (1 dòng chữ) → Khối tự co lại siêu mỏng ôm sát dòng chữ.



(b) Trường hợp 2: Bổ sung thêm dòng nội dung → Khối tự động nới rộng chiều cao theo nội dung.

Hình 2.16: Thực Nghiệm Trực Quan: Chiều Cao Thực Tế Của Khối Co Giãn Theo Nội Dung Dù Đang Đặt `max-height: 300px;`

2.4.4 Cạm Bẫy Thực Chiến: Xung Đột Giữa `height` Cố Định Và `max-height`

Một lỗi rất phổ biến mà nhiều lập trình viên mới gặp phải là khai báo đồng thời cả `height` và `max-height` cùng một con số:

[Cảnh Báo] Cạm Bẫy: Vô Hiệu Hóa max-height Khi Dùng Chung Với height Cố Định

```
.MAX {
    background: red;
    width: 500px;
    height: 300px;      /* Khoa cung chieu cao 300px */
    max-height: 300px;  /* Gioi han tran 300px -> HOAN TOAN VO TAC DUNG! */
    font-size: 20px;
}
```

Nguyên nhân: Khi bạn đã ấn định `height: 300px;`, phần tử đã bị khóa cứng ở 300px. Do đó, việc đặt thêm `max-height: 300px;` là hoàn toàn dư thừa vì phần tử không còn khả năng co giãn nữa!

Giải Pháp Chuẩn Kỹ Thuật: Sử Dụng `height: auto;`

Để `max-height` phát huy toàn bộ sức mạnh, bạn phải cho phép phần tử được tự do co giãn bằng cách thiết lập `height: auto;` (hoặc không khai báo thuộc tính `height`):

Giải Pháp Khắc Phục Chuẩn Kỹ Thuật

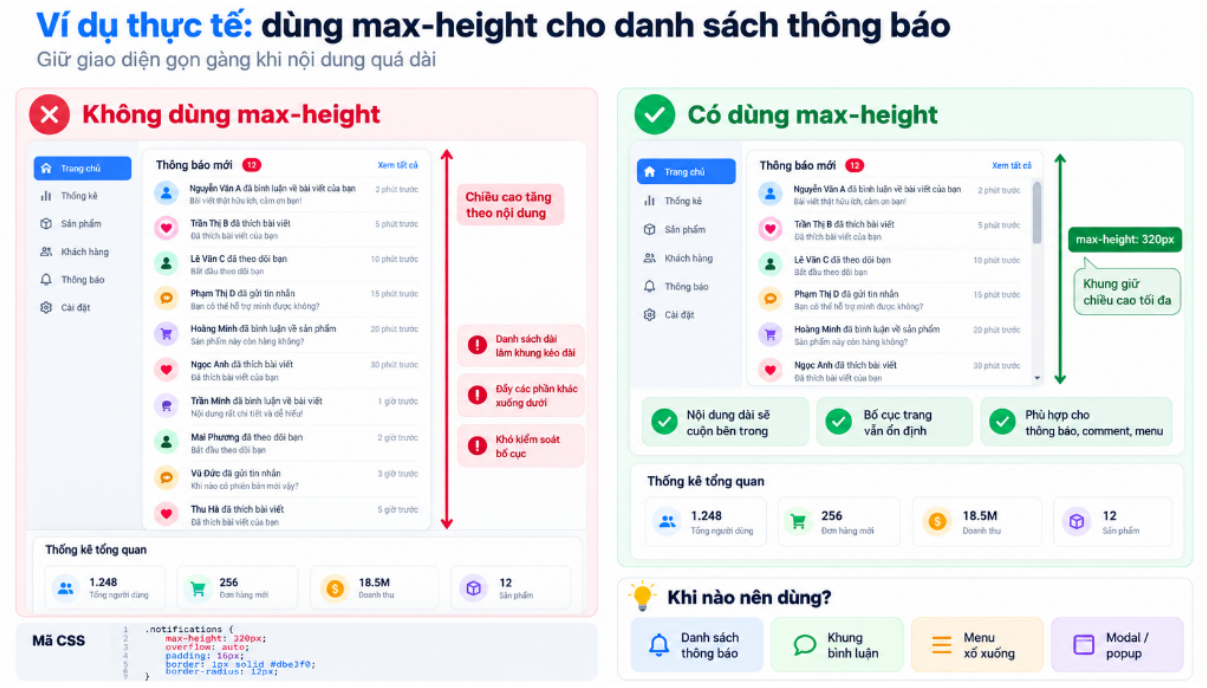
```
.MAX {
    background: red;
    width: 500px;
    max-height: 300px;  /* Tran chieu cao toi da 300px */
    height: auto;       /* Co gian tu do theo noi dung */
    font-size: 20px;
    overflow: auto;     /* Tu dong cuon neu vuot qua 300px */
}
```

- **Nếu nội dung ít**: Khối `.MAX` sẽ nhỏ hơn 300px, ôm sát chữ.
- **Nếu nội dung nhiều**: Khối `.MAX` dừng lại ở 300px và sinh thanh cuộn dọc mượt mà.

2.4.5 Các Ứng Dụng Thực Tế Tiêu Biểu Của max-height

Ứng Dụng 1: Kết Hợp Với overflow Tạo Khung Cuộn Danh Sách Thông Báo

Khi xây dựng giao diện bảng điều khiển (**Dashboard**), danh sách thông báo hoặc khung bình luận thường có độ dài biến động. Nếu không giới hạn chiều cao, danh sách dài sẽ đẩy toàn bộ nội dung khác xuống dưới làm vỡ bố cục trang:



Hình 2.17: Ví Dụ Thực Tế: Ứng Dụng `max-height: 320px;` Giữ Khung Thông Báo Luôn Gọn Gàng & Ổn Định Bố Cục Trang

Mã Nguồn Khung Thông Báo Chuẩn Dashboard Với max-height

```
.notifications {
    max-height: 320px;
    overflow: auto;
    padding: 16px;
    border: 1px solid #dbe3f0;
    border-radius: 12px;
}
```

Ứng Dụng 2: Tạo Hiệu Ứng Mở Rộng / Thu Gọn Mượt Mà (Accordion Transition)

Trong CSS, bạn **không thể** áp dụng hiệu ứng chuyển động (`transition`) trực tiếp từ `height: 0;` lên `height: auto;`. Cách giải quyết kinh điển là kết hợp `max-height` và `transition`:

Tạo Hiệu Ứng Accordion Mượt Mà Không Cần JavaScript

```
.accordion-content {
    max-height: 0;                          /* Ban dau thu gon hoan toan */
    overflow: hidden;                       /* An phan noi dung bi thu gon */
    transition: max-height 0.4s ease-in-out;/* Hieu ung chuyen dong muot ma */
}

/* Khi hover hoac khi co class kich hoat */
.accordion-item:hover .accordion-content {
    max-height: 400px;                      /* Mo rong den nguong du chua noi
    dung */
}
```

2.4.6 Khi Nào Nên Dùng `max-height` & Lưu Ý Thực Chiến

[Mẹo] 4 Trường Hợp Vàng Nên Sử Dụng max-height

1. **Giới hạn chiều cao an toàn**: Không làm mất đi tính co giãn linh hoạt tự nhiên của phần tử.
2. **Hiển thị thanh cuộn tự động**: Xử lý các khối nội dung người dùng sinh ra (**User Generated Content**) có độ dài khó lường.
3. **Hiệu ứng chuyển động**: Làm Accordion, Dropdown Menu, Collapsible Panels bằng CSS thuần.
4. **Tối ưu hóa Responsive**: Giữ các khối hình ảnh/video không vượt quá độ cao màn hình điện thoại (ví dụ: `max-height: 80vh;`).

[Cảnh Báo] 3 Lưu Ý Sống Còn Khi Dùng max-height

- **Không đặt** `max-height: none;` nếu đang muốn hạn chế chiều cao, vì `none` là giá trị hủy bỏ giới hạn.
- **Cẩn thận với** `overflow: hidden;`: Nếu nội dung dài hơn `max-height`, toàn bộ phần tràn sẽ bị cắt cụt vĩnh viễn và người dùng không thể đọc được nếu không có cơ chế mở rộng.
- **Khi làm hiệu ứng transition**: Tránh chuyển từ `max-height: 0` sang `none`; thay vào đó hãy đặt một giá trị cụ thể ước lượng vừa đủ lớn (ví dụ: `500px` hoặc `1000px`).